CONTENT MONETIZATION MODELER

YouTube revenue prediction

Project guide and complete code walkthrough

Python · Data visualization · Machine learning · Streamlit

Learn what the project does, how every file works, why each decision was made, and how to explain the results.

Includes setup instructions, all project source code with numbered explanations, notebook guidance, charts, application screenshots, tests and viva preparation.

Final model: Linear Regression Reserved test: MAE $3.17, RMSE $13.61, R^2 0.9516

Synthetic learning dataset. Predictions concern one reporting observation.

Prepared 3 November 2026

Contents

1. Project and problem statement 4

2. Install, open and run 5

3. Dataset and its limitations 6

4. Preparation and prevention of leakage 7

5. Understand the five algorithms 8

6. Metrics and actual results 9

7. Read the charts 10

8. Streamlit behavior and error handling 11

9. Project structure and generated artifacts 12

10. Testing and troubleshooting 13

11. Presentation and viva preparation 14

12. Visual results 15

13. Application screenshots 20

14. Python reading key 24

15. Code walkthrough: utils.py 25

16. Code walkthrough: train_model.py 32

17. Code walkthrough: charts.py 37

18. Code walkthrough: app.py 41

19. Notebook cell-by-cell guide 51

Notebook cell 3: 1. Imports and project folder 52

Notebook cell 5: 2. Load and validate the data 53

Notebook cell 7: 3. Missing values and synthetic-data issues 54

Notebook cell 9: 4. Separate training, validation and test data 55

Notebook cell 11: 5. Numerical exploration on the training partition 56

Notebook cell 13: 6. Relationships, groups and reporting dates 57

Notebook cell 15: 7. Two understandable engineered features 58

Notebook cell 17: 8. Preprocessing and five regression models 59

Notebook cell 19: 9. Read the errors correctly 60

Notebook cell 21: 10. Visual analysis and model diagnostics 61

Notebook cell 23: 11. Feature importance without using test data for selection 62

Notebook cell 25: 12. Load the saved pipeline and predict 63

Notebook cell 27: 13. Missing inputs and rejected extrapolation 64

20. Automated tests: complete code 65

21. Notebook execution helper 73

22. Launcher, dependencies and configuration 74

23. Sources and final limitations 76

Suggested reading order

First read chapters 1–6 and open the application. Then use the charts and interface chapters. Read utils.py before train_model.py, followed by charts.py and app.py. Treat the numbered source commentary as a reference while stepping through each file.

Line numbers in the code chapters refer to the delivered source files. A statement spanning several lines receives one explanation for that complete statement. Blank lines separate ideas; comment and docstring lines already explain themselves. Indentation tells you which statements belong inside a function, condition, loop or context.

1. Project and problem statement

Content Monetization Modeler estimates YouTube advertising revenue in US dollars from performance and context for one reporting observation. This is supervised regression: historical rows contain input measurements and a known numeric revenue target. Classification would predict a label, such as a category, which is not the task here.

The user supplies views, likes, comments, total watch time, video length, subscriber count, category, device and country. The app prepares these values exactly as training did and calls the fitted model. It also provides descriptive visualizations and model diagnostics. There is no live YouTube connection, automatic video upload, or automatic forecasting of views.

Business uses suggested by the brief include scenario planning for creators and media companies. In this implementation they are demonstrations using synthetic learning data. The estimate concerns the given reporting observation, not the lifetime income of a video. If a scenario concerns a future video, the performance figures are assumptions supplied by the user.

The original brief names Linear Regression and also requests a comparison of five regression models. Both are fulfilled. Linear Regression is a candidate and the final selected model. The selection follows a predefined validation rule, not a decision based on the test score.

The teaching folders contain generic Streamlit and breast-cancer classification examples. We used their concepts, such as forms and saving a model, while creating a separate revenue regression project. Classification metrics, a SQL registration screen, OCR and unrelated advanced methods were not imported into the requirements.

2. Install, open and run

Extract the entire project ZIP. The model, original CSV, metadata and reports must remain beside the Python files in their supplied folders. Python 3.12, 64-bit, is the tested language version. Python can be installed from https://www.python.org/downloads/. The first package installation needs an Internet connection.

On Windows, START_APP.bat is a convenience launcher. It changes to its own folder, looks for a local .venv, creates one using Python 3.12 when necessary, installs requirements.txt and runs Streamlit on the loopback address. If setup fails it displays a message and pauses so the error can be read. The manual commands below are equally valid.

Open PowerShell in the project folder. Run: py -3.12 -m venv .venv. Then run: .\.venv\Scripts\python.exe -m pip install -r requirements.txt. Finally run: .\.venv\Scripts\python.exe -m streamlit run app.py. Open the URL printed in the terminal, usually http://localhost:8501. Keep the terminal running while using the app. Ctrl+C stops it.

If the py launcher is unavailable but python --version reports Python 3.12, replace py -3.12 with python for environment creation. No PowerShell activation-policy change is needed: the commands call the virtual environment's Python directly. A virtual environment keeps the project's packages separate from other projects.

On macOS or Linux, use python3.12 -m venv .venv, then .venv/bin/python -m pip install -r requirements.txt and .venv/bin/python -m streamlit run app.py. These instructions are provided for portability; verification was performed on Windows rather than those platforms.

The trained model is already supplied. To rebuild it, run the environment's Python with train_model.py. To run tests, use python -m pytest tests -q with that same interpreter. To execute the notebook from a fresh kernel, run python tests/run_notebook.py. The notebook itself also retrains the five models and regenerates outputs.

Open notebooks/project_analysis.ipynb in VS Code with notebook support, or Jupyter, and select the project environment as its kernel. The notebook contains 28 cells, including 13 executable cells, with explanations between them. Running all cells gives a complete, reproducible project walkthrough.

3. Dataset and its limitations

The original CSV has 122,400 rows and 12 columns. Removing 2,400 exact duplicate rows leaves 120,000 observations and 5,000 video IDs. Dates span 9 June 2024 to 8 June 2025. The numeric target is ad_revenue_usd. The source was supplied as synthetic data for learning.

After exact duplicate removal, likes, comments and watch_time_minutes each have exactly 6,000 missing values, or 5%. The remaining columns have no missing values. A missing value is unknown; zero would be an observed value and must not be used as an automatic replacement.

Views range only from 9,521 to 10,468. Revenue ranges from about $126.59 to $382.77 per observation. These ranges describe the supplied sample, not all YouTube videos. The app rejects inputs outside the numeric ranges used to fit the final model. Continuous limits are rounded inward to two decimals, so the displayed limits are accepted values.

Exact duplicates are not the same as repeated IDs. There are 3,849 extra rows sharing a video/date pair after exact deduplication. Different contextual observations can share such a key. All 5,000 IDs also have multiple categories and lengths, so the synthetic identifier is not a reliable real-world video identity. We preserve distinct rows and state this limitation rather than merging unrelated measurements.

In 7,280 deduplicated observations, total watch time divided by views exceeds video length. Replays or metric definitions might explain some cases, but the source does not settle the issue. The project does not label this ratio as a verified retention percentage, and the app warns when an entered combination has this property.

The 1.5-IQR rule flags unusual values rather than proving errors. In the full deduplicated source it flags 865 view counts, all still plausible positive values. No valid row was discarded merely to improve a score. The training notebook recalculates its own IQR flags using training rows.

Watch time and revenue have an observed Pearson correlation of approximately 0.9889 among deduplicated rows with watch time present. This is descriptive evidence, not proof of causation. Groups such as country and device have small differences in average revenue in this source; strong real-world claims about them would be unsupported.

Column dictionary

Column	Meaning / unit
video_id	Synthetic identifier; used for split and audit, not prediction
date	Reporting timestamp; used for descriptive date summaries
views	View count for the observation
likes	Like count; optional if unknown
comments	Comment count; optional if unknown
watch_time_minutes	Total watch time in minutes, not average; optional if unknown
video_length_minutes	Video duration in minutes
subscribers	Channel subscriber count
category	Education, Entertainment, Gaming, Lifestyle, Music or Tech
device	Desktop, Mobile, TV or Tablet
country	AU, CA, DE, IN, UK or US
ad_revenue_usd	Observed revenue in US dollars; prediction target

4. Preparation and prevention of leakage

Data leakage occurs when model training or selection uses information that would not be available in a fair evaluation. Examples include calculating imputation medians from the final test rows, selecting the best model by repeatedly inspecting test errors, or using a feature computed from the target revenue.

This project first separates IDs into training, validation and test partitions. Training contains 3,000 IDs and 72,176 rows. Validation contains 1,000 IDs and 23,967 rows. Test contains 1,000 IDs and 23,857 rows. GroupShuffleSplit keeps each video ID in one partition, using seed 42. The requested 60/20/20 split applies to ID counts, so row counts are not exactly those percentages.

During the five-model comparison, fit is called only on training rows. Each candidate therefore learns numeric medians, categorical replacement values, category encodings, scaling parameters and model coefficients or trees only from training. Validation predicts through those already-fitted steps. Test is not used to choose the candidate.

After selection, a fresh copy of the selected algorithm is fitted on training plus validation. At that point it is appropriate to learn preparation from both development partitions. The reserved test IDs remain excluded. The final saved pipeline is this refitted model, and its reported test metrics come from the same saved behavior.

A scikit-learn Pipeline runs preparation stages in a fixed order. FunctionTransformer calls the ordinary add_features function. ColumnTransformer sends numeric and categorical columns through their appropriate transformations. Keeping these stages together is slightly more structured than manual fillna calls, but it prevents the app from accidentally using different medians, columns or scaling.

Numeric missing values use the median: the middle value after ordering the observed training values. Missing indicators add binary columns for fields that had missing values at fit time. StandardScaler centers and scales numeric values using fitted means and standard deviations. Trees do not require scaling, but using one common path makes the beginner project simpler and keeps the comparison consistent.

Categorical missing values use the most frequent fitted category. OneHotEncoder makes indicator columns for categories, devices and countries. It does not invent an ordinal relationship between, for example, India and Canada. handle_unknown='ignore' makes an unseen category safe at the pipeline level; the public prediction helper still requires a known choice.

The full set of indicator columns can make some linear coefficients non-unique because of redundant category columns and correlated inputs. Predictions remain usable. We use permutation importance for the explanation rather than treating every raw coefficient as an independent causal effect.

Engagement rate equals (likes + comments) / views. For 1,000 likes, 200 comments and 10,000 views, the rate is 0.12, or 12%. Average watch minutes equals total watch minutes / views. For 30,000 minutes and 10,000 views it is 3 minutes per view. Zero denominators become missing results instead of infinity.

Ratios are calculated before imputation. If a necessary raw input is missing, its ratio is also missing. The ratio receives its own fitted median and its missingness can be flagged. As a result, an imputed ratio need not equal the ratio of separately imputed raw values. This behavior is consistent in training, prediction and the unscaled cleaned-data export.

The cleaned CSV is an analysis export with imputed values, original-missing flags and partition labels. Its preparation comes from the final training-plus-validation pipeline. Train from the original CSV, not this export, to preserve original missingness. The app's exploratory charts use deduplicated source values before imputation; the notebook's detailed development exploration uses training rows.

5. Understand the five algorithms

Linear Regression predicts a numeric output from a weighted sum of input features plus an intercept. A simplified expression is predicted revenue = intercept + weight_1 * feature_1 + weight_2 * feature_2 + ... In our actual pipeline the features have already been encoded and scaled. Fitting chooses weights that minimize squared residuals. It is the required baseline and easiest final model to explain.

Ridge Regression also uses a weighted sum, but adds a penalty for large coefficient values. alpha=10.0 controls that penalty in this project. Regularization can stabilize coefficients when features are correlated. It does not automatically guarantee a better validation result or make correlations causal.

A Decision Tree repeatedly divides observations according to feature thresholds, then predicts a leaf's learned value. max_depth=8 limits the number of successive splits. min_samples_leaf=20 requires at least 20 training observations in each leaf. These settings reduce the risk of memorizing very small groups.

A Random Forest averages many trees, each trained with randomness. This project uses 80 trees, depth at most 12, and at least 8 samples per leaf. Averaging reduces the instability of a single tree. n_jobs=2 allows two CPU workers; random_state=42 controls reproducibility. The forest has lower training error than the simpler models here, but that alone is not a reason to select it.

Gradient Boosting adds small trees sequentially, each helping correct previous errors. We use 100 trees, maximum depth 2 and at least 20 observations per leaf. The default learning rate is 0.1. This can learn nonlinear patterns, although the supplied source is already well represented by a linear model.

These are fixed, modest choices. There is no claim of exhaustive hyperparameter tuning or a universally best algorithm. All candidates use the same split and preprocessing strategy so differences are easier to assess. Linear Regression is earlier than Ridge in the explicit simplicity order, followed by the tree, forest and boosting models.

Ridge has validation RMSE about $13.8082 and Linear Regression about $13.8101. Their difference is about $0.0019. The predefined selection rule allows a simpler candidate within $0.05 of the minimum RMSE, so Linear Regression is selected. It also has lower validation MAE. The test set did not decide this choice.

6. Metrics and actual results

A residual is actual revenue minus predicted revenue. For actual values [100, 200, 300] and predictions [110, 190, 320], residuals are [-10, 10, -20]. Their absolute values are [10, 10, 20]. MAE is (10 + 10 + 20) / 3 = 13.33 dollars.

For the same example, squared residuals are [100, 100, 400]. RMSE is the square root of their mean: sqrt(600 / 3) = 14.14 dollars. The squaring makes large mistakes matter more. Both MAE and RMSE have the same units as the target, which is USD here.

R-squared is 1 - sum of squared residuals / sum of squared deviations of actual values from their mean. In the example, the actual mean is 200, the denominator is 20,000 and the numerator is 600, so $R^2 = 0.97$. R^2 can be negative. It is not a classification accuracy percentage and should not be described as '97% of predictions are correct'.

The final model's reserved test MAE is $3.1741, RMSE is $13.6116 and R^2 is 0.9516, on 23,857 observations. A simple mean-only prediction, using the development target mean, has test MAE $53.2698. That reference checks whether the learned model improves on an uninformative constant prediction.

For 22,660 test rows where watch time is observed, MAE is $0.6121 and RMSE is $2.0283. For 1,197 rows where watch time is missing, MAE is $51.6736 and RMSE is $60.1231. The difference is large because imputation cannot recover the main information in an unknown watch-time value. This also explains why aggregate RMSE is much higher than MAE.

A low average error does not guarantee a specific person's next prediction. These figures are empirical results on synthetic test observations. The app shows the appropriate watch-time subgroup's historical MAE as context, and explicitly does not treat it as a confidence interval.

Permutation importance shuffles one original input on a validation sample, recomputes its derived features through the pipeline and measures the increase in MAE. Three repetitions provide a mean and a standard deviation. This is done before the final refit, using 1,500 validation rows. Larger positive increases indicate greater predictive reliance. Slightly negative values can occur through sample variation.

Model	Validation MAE ($)	Validation RMSE ($)	Validation R^2
Linear Regression	3.2371	13.8101	0.9506
Ridge Regression	3.2890	13.8082	0.9506
Decision Tree	5.2908	14.1681	0.9480
Random Forest	3.6774	13.8766	0.9501
Gradient Boosting	4.1122	13.9009	0.9499

7. Read the charts

The missing-value chart audits all deduplicated rows and identifies the three incomplete inputs. Distribution charts show that views occupy a narrow band, while watch time and revenue span broader ranges. Box plots show the center and spread without making every flagged observation an error.

The watch-time scatter plot shows a strong relationship with revenue. The views scatter has a narrow horizontal range and a much weaker relationship in this source. Both display a reproducible sample so points remain visible. Model metrics still use the full intended partitions.

Category, device and country comparisons use mean revenue. Horizontal bar axes begin at zero to avoid exaggerating tiny differences. The correlation heatmap summarizes pairwise linear relationships; it does not describe causal effects or automatically justify removing every correlated feature.

Monthly charts show mean revenue per observed row. Counts vary by month, and the first and last months are partial. The series does not establish channel growth or reliable seasonality. It is a descriptive summary of the sample and should be presented that way.

The validation comparison chart uses RMSE, where lower is better. Its accompanying table contains training and validation MAE, RMSE and R^2. Looking at both training and validation helps identify when a more complex model fits training better without improving unseen observations.

In the actual-versus-predicted chart, perfect predictions would lie on the diagonal. The residual chart plots actual minus predicted revenue around a horizontal zero line. A broad vertical band near the central predictions is consistent with missing-watch-time rows being estimated near a typical value. The subgroup metrics substantiate that explanation.

The feature-importance figure measures validation MAE changes before final refitting. Watch time is the dominant input in this dataset, with likes contributing some additional information. The plot does not mean that a creator can change one input independently of all others to guarantee a revenue increase.

8. Streamlit behavior and error handling

Streamlit runs the Python app from top to bottom when a user interacts with controls. A sidebar radio chooses which page function to call. Standard widgets create the interface, so there is no separate JavaScript application to explain. A form groups prediction inputs and submits them together with the Predict revenue button.

cache_data stores the loaded data result between reruns. The source file's modification timestamp is part of the cache key, so changing the file refreshes it. cache_resource stores the fitted model resource. Model and metadata timestamps trigger refresh after regeneration. Training is never performed merely because a user changes a widget.

The model loader compares the model's SHA-256 hash with its metadata and checks the exact scikit-learn, NumPy, Pandas and joblib versions. The hash catches a mismatched or accidentally altered artifact. It is not a substitute for a trusted source: an attacker who changes both files could also change the hash. No arbitrary model upload is offered.

The prediction helper validates required fields, finite non-negative numbers, integer counts, known categories and supported ranges. Missing likes, comments or watch time are accepted only as unknown values. It rejects unsupported extrapolation instead of silently clipping input or output. Combinations inside individual ranges can still be unusual, a limitation stated on the page.

Missing model files, incompatible versions, absent charts and invalid source files produce recovery messages. An incomplete date range asks the user to pick both dates. A filter with no matches produces an informative message instead of calculating averages of an empty table. The app's report-image helper checks for absent image files before rendering them.

The prediction JSON download contains the supplied input values, selected model name and estimated revenue. The filtered CSV download contains all matching rows, while the preview table shows only the first 200. Predictions are not written to a user-history database. Standard Streamlit reruns may clear the displayed result after another interaction; pressing Predict generates it again.

9. Project structure and generated artifacts

app.py defines the four Streamlit screens and routes navigation. utils.py contains reusable data validation, splitting, feature creation, pipeline construction and prediction checks. train_model.py compares five estimators, refits the selected model and creates the final outputs. charts.py writes the ten static PNG figures used by reports and the PDF.

notebooks/project_analysis.ipynb narrates the project and calls the same shared functions. tests/test_project.py contains regression checks. tests/run_notebook.py starts a fresh kernel with the current interpreter and executes every notebook cell. Sharing functions prevents the notebook and app from drifting into different preprocessing rules.

data/youtube_ad_revenue_dataset.csv is the unchanged source. data/cleaned_youtube_data.csv is the descriptive export. models/revenue_pipeline.joblib stores the fitted pipeline. models/metadata.json describes its input ranges, defaults, categories, package versions, partition counts and file hashes.

reports/model_comparison.csv contains all five training and validation results. reports/test_metrics.json contains the selected model's final test scores, a mean baseline and watch-time subgroups. reports/test_predictions.csv stores actual and predicted test values. reports/data_partitions.csv maps every video ID to one partition. reports/feature_importance.csv stores the validation permutation results, and reports/data_quality.json records cleaning counts.

reports/figures holds the ten PNGs. docs/screenshots contains browser captures. This PDF and its text companion explain the final files. reports/verification.md records checks and limitations. requirements.txt lists pinned direct dependencies; requirements-lock.txt records the tested complete environment. .streamlit/config.toml supplies the light theme and disables usage statistics.

The local environment and cache folders are excluded from the ZIP and Git rules. Extracting the package therefore does not transplant an interpreter from another computer. Install packages into a new environment using the instructions in the README.

10. Testing and troubleshooting

The automated suite checks the supplied row and duplicate counts, absence of ID overlap, deterministic partitions, rejected corrupt input, feature formulas, fitted medians, absence of target leakage, prediction consistency after saving/loading, shuffled input-column order, unseen categories at the pipeline level, optional missing inputs, displayed range endpoints and the completeness of the cleaned export.

Streamlit tests open all four screens, submit valid and unsupported inputs, mark watch time unknown, change filters, check an incomplete date range and simulate a missing model in an isolated temporary directory. These tests use the real app script, not a separate imitation of its logic. The final suite contains 44 passing test cases.

Fresh-kernel notebook execution runs all 28 cells, including full retraining, without a cell error. Repeated training reproduced the same test metrics. Browser checks at desktop size verify charts, predictions and both download types without JavaScript exceptions. A fresh narrow viewport checks layout overflow. These checks do not replace physical-device or multi-user load testing.

Use the same Python environment for package installation, training, tests and Streamlit. If a package is missing, rerun pip install -r requirements.txt with that environment's interpreter. If a saved model does not match its metadata, regenerate it with train_model.py. A missing report or figure can be rebuilt the same way.

If the browser cannot connect, check that the terminal is still running and use its printed address. If port 8501 is occupied, add --server.port 8502. If the data file is absent, restore it from the supplied ZIP or original source. If a notebook cannot find utils.py, keep it in the project's notebooks folder or launch it from the project root.

The tested environment is Windows with Python 3.12 and the pinned libraries. macOS/Linux, public cloud deployment, simultaneous-user load, adversarial security testing and real YouTube data were not validated. No finite test suite can guarantee the absence of every future error. The delivered checks target the actual project behavior and known edge cases.

11. Presentation and viva preparation

A 12-minute project explanation can use this sequence: one minute for the problem and regression target; two minutes for the dataset, missing values and synthetic limitations; two minutes for preparation, features and data separation; two minutes for the five models and selection; two minutes for results and subgroup errors; two minutes for the live Streamlit demo; and one minute for limitations and future work.

Why regression? The target is continuous USD revenue. Why Linear Regression? It meets the brief, is easy to explain and falls within the predefined five-cent RMSE tolerance of the best validation candidate. Why not automatically choose the forest? It has lower training error but does not improve validation enough to justify selection.

Why median imputation? It is simple, robust to extreme numeric values and works inside a fitted pipeline. Why keep missing indicators? The model can recognize that a value was originally unknown. Why is missing watch time difficult? It removes the strongest measured revenue predictor; a median cannot restore the lost individual information.

Why one-hot encode categories? A text category has no natural numeric ranking. Why split by video ID? It prevents the same identifier from appearing in training and evaluation. Why still state a limitation? The synthetic identifiers are inconsistent and the split does not prove generalization to real channels or future periods.

What is the difference between MAE and RMSE? Both measure dollar error, but RMSE gives large errors more weight. What does R^2 mean? Performance relative to a mean reference; it is not percent accuracy. Why show subgroups? The overall average hides much larger errors when watch time is missing.

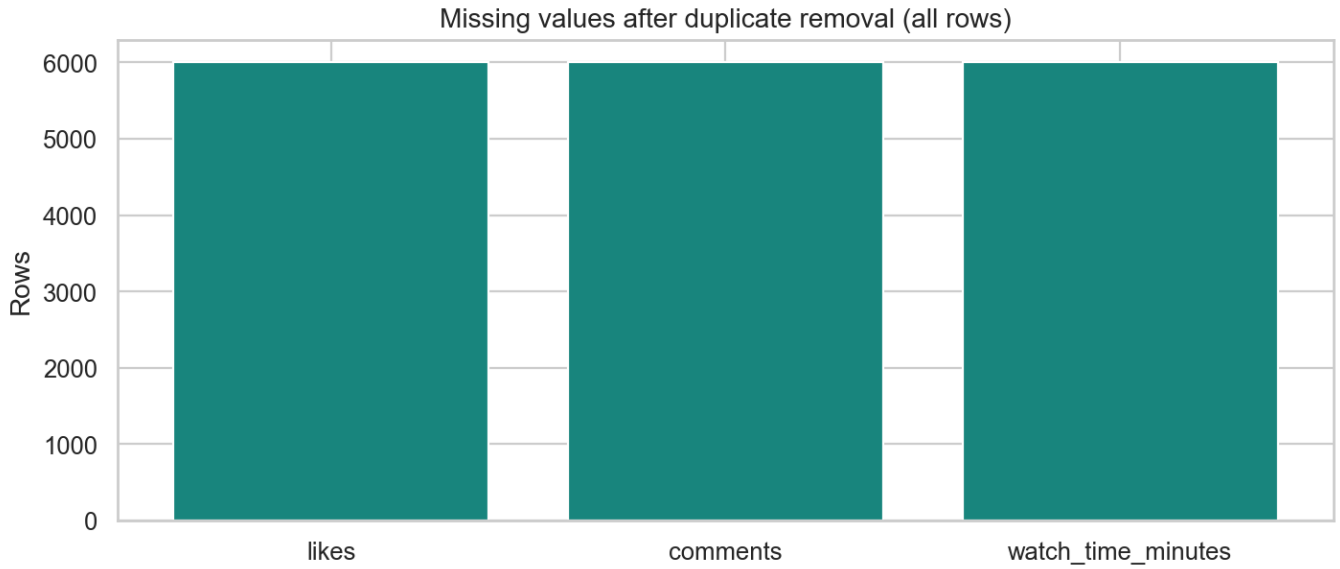
What does pickling mean? Converting Python objects into bytes and restoring them later. Joblib is convenient for storing scikit-learn objects, but its model files must come from trusted sources. Why save preparation with the model? It preserves column handling, fitted medians, category encoding and scaling during app prediction.

What would improve the project? Representative real-world data, stable video/channel identifiers, clear reporting definitions, wider view ranges, stronger watch-time completeness and evaluation on future dates and unseen channels. Confidence intervals would require an additional calibrated uncertainty method; the present app does not invent them.

Be able to trace one example from the form through predict_revenue, add_features, the preparation transformer, Linear Regression and the displayed result. Explain the code in your own words. The next chapters include the exact files with source line numbers and explanations of executable statements.

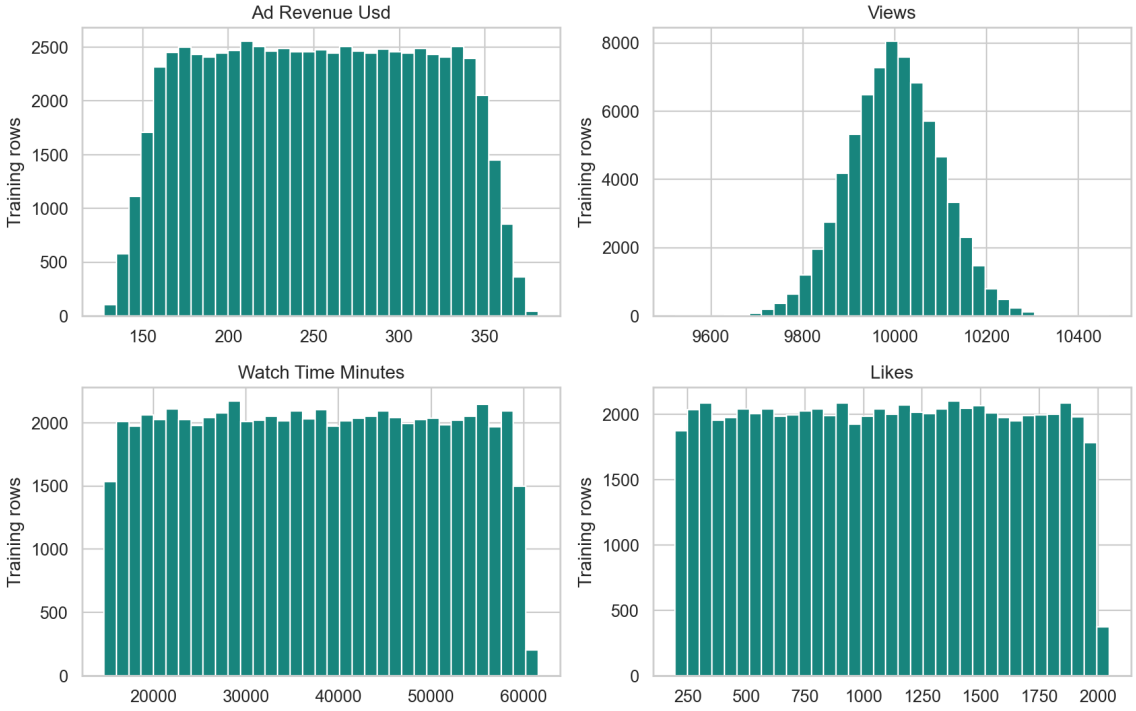
12. Visual results

01 missing values



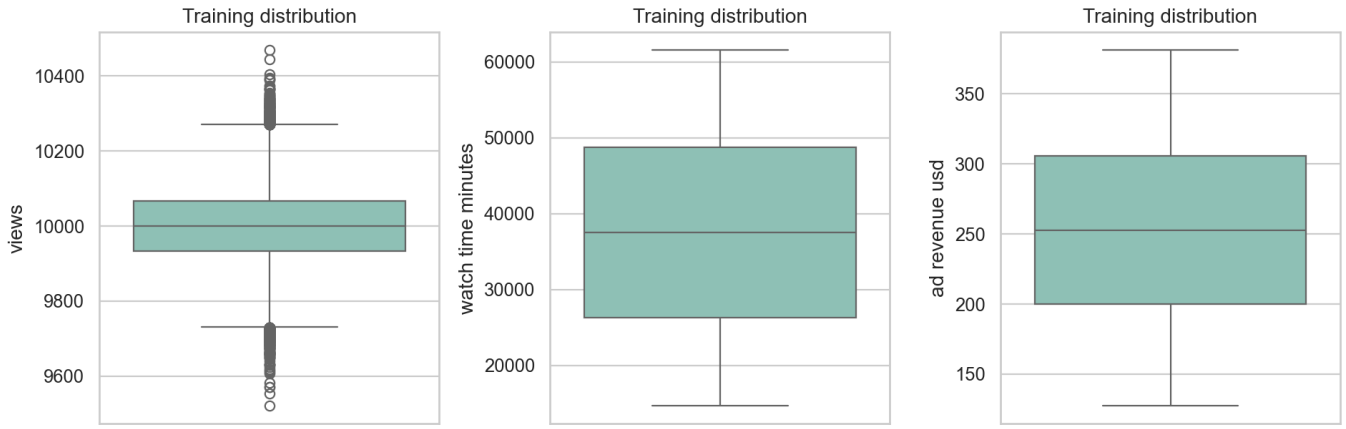
All three incomplete source inputs have 6,000 missing values after duplicate removal.

02 distributions



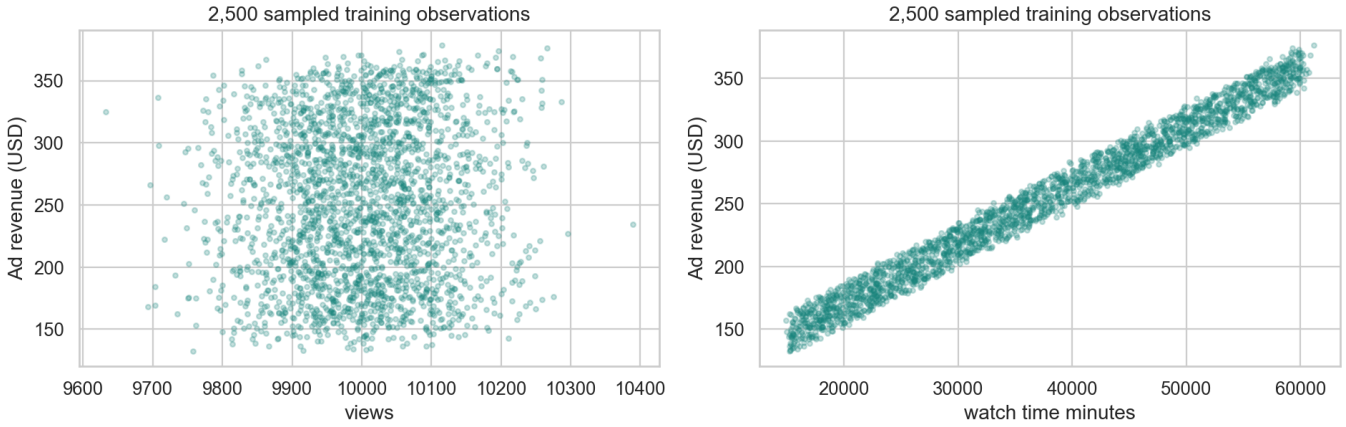
Training distributions reveal the narrow view range and broader watch-time/revenue variation.

03 outliers



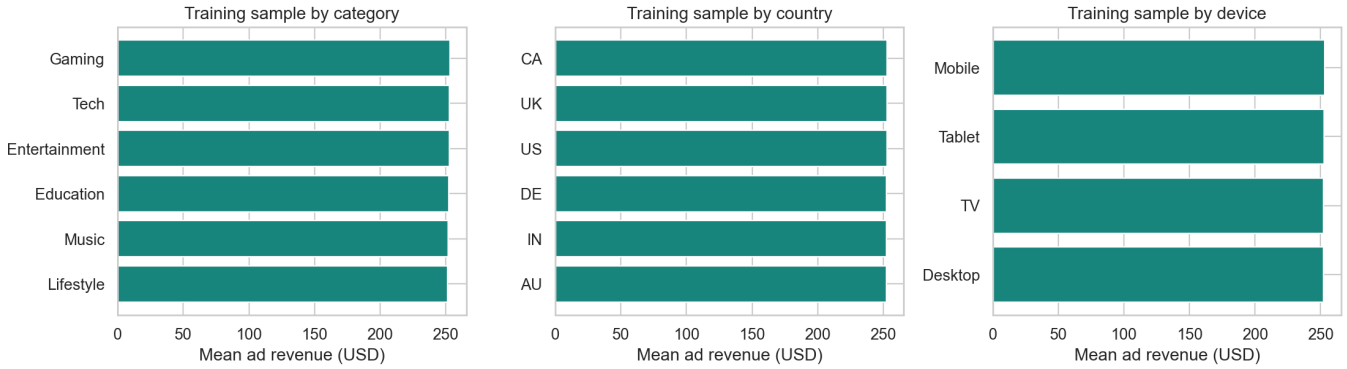
Box plots support inspection. A point outside an IQR whisker is not automatically deleted.

04 revenue relationships



Watch time has a strong observed relationship with revenue; views vary within a narrow band.

05 group comparisons



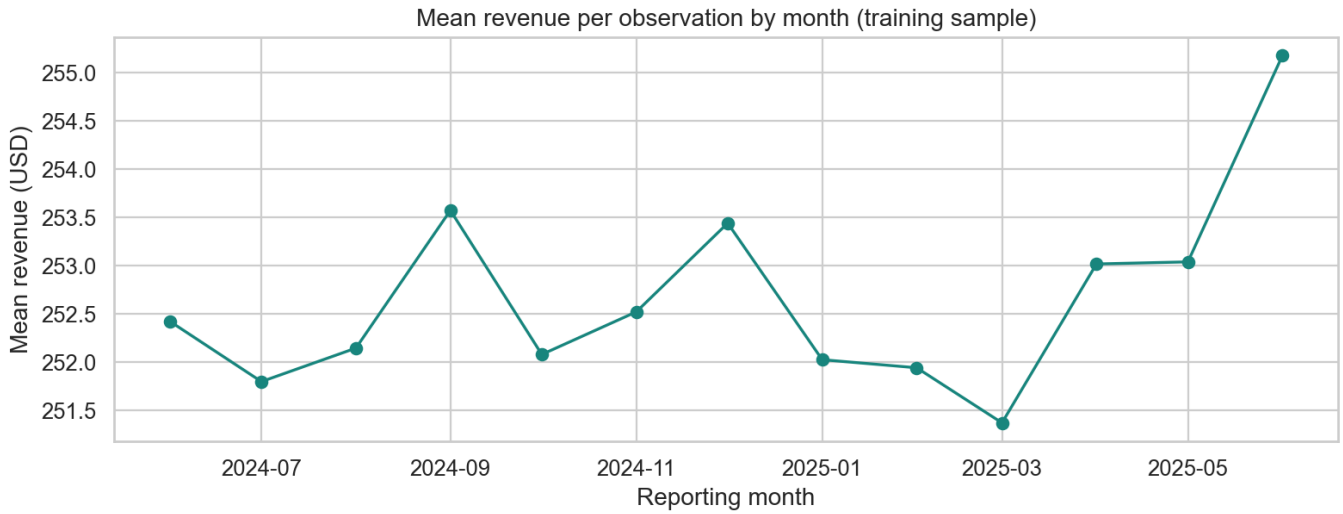
Group means differ only slightly. Axes begin at zero to avoid overstating the differences.

06 correlations



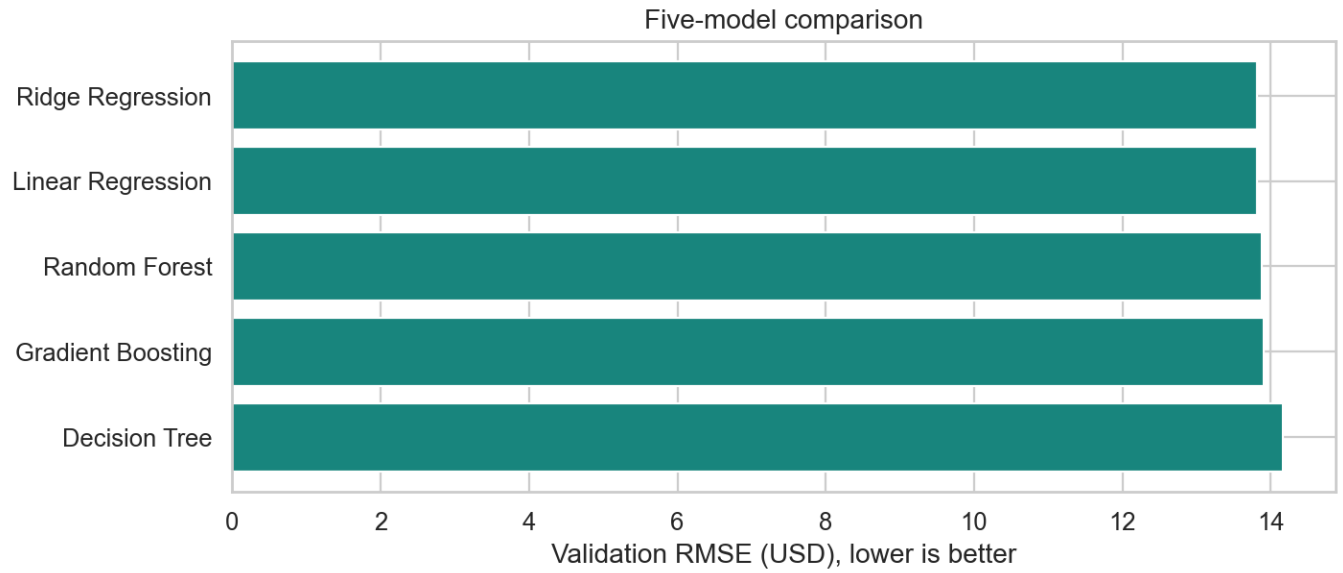
Training correlations describe pairwise linear relationships and do not establish causation.

07 monthly sample



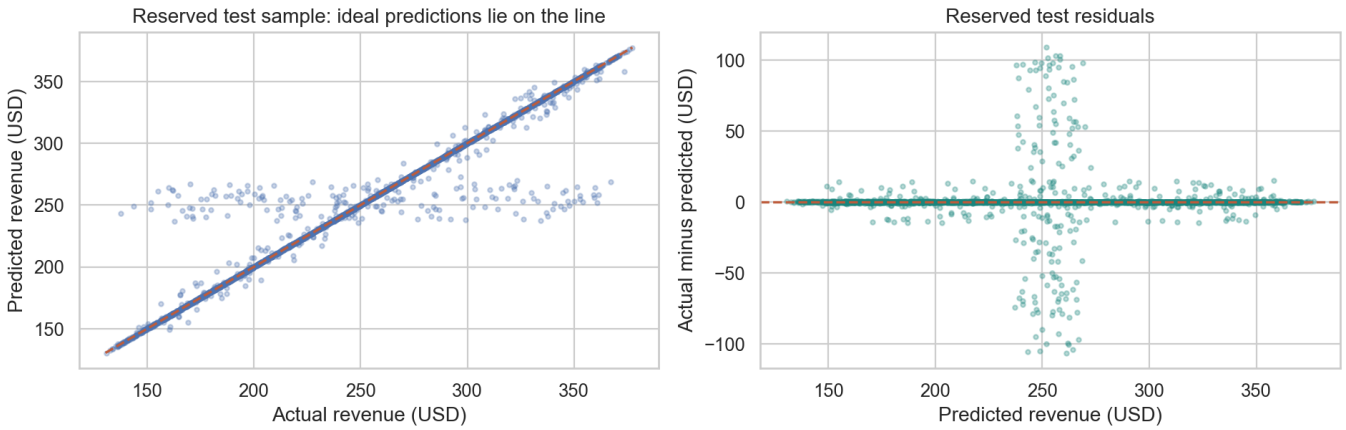
Monthly sample averages are descriptive. The first and last months have partial coverage.

08 model comparison



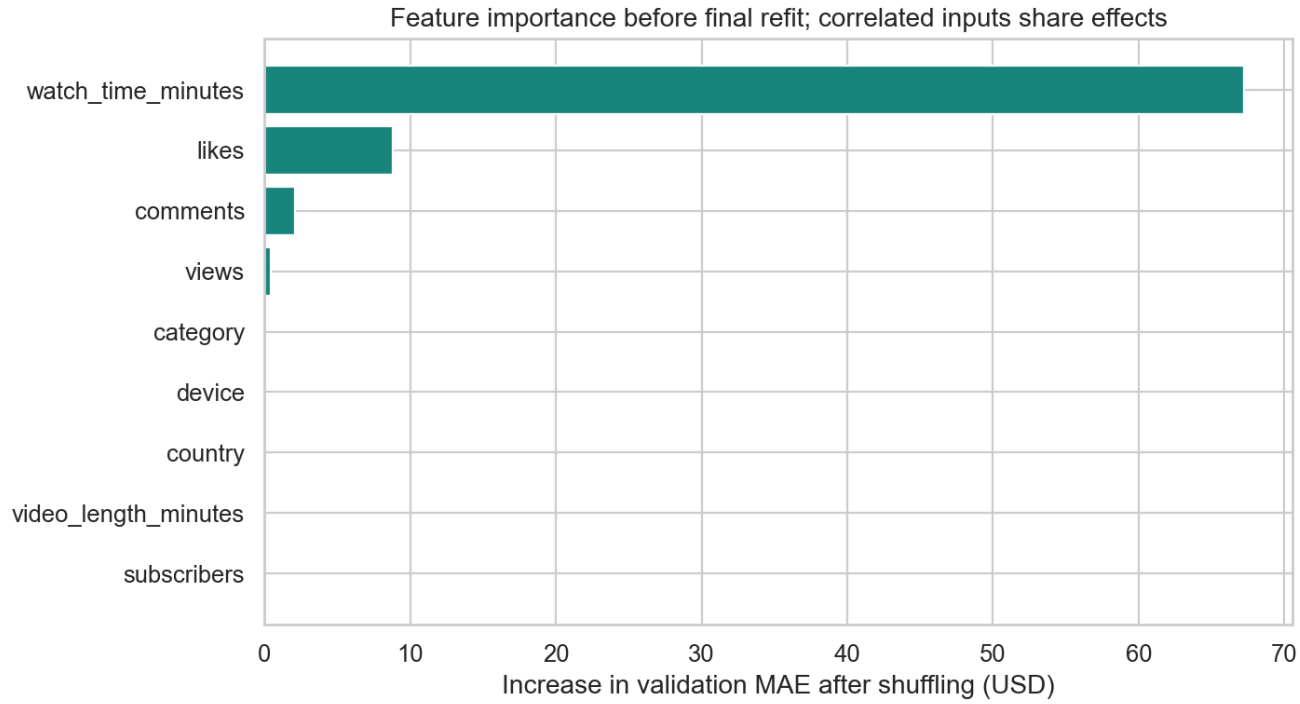
The two linear models have almost equal validation RMSE. The predefined rule selects Linear Regression.

09 prediction errors



Reserved test errors show a central band of less informative predictions, consistent with missing watch time.

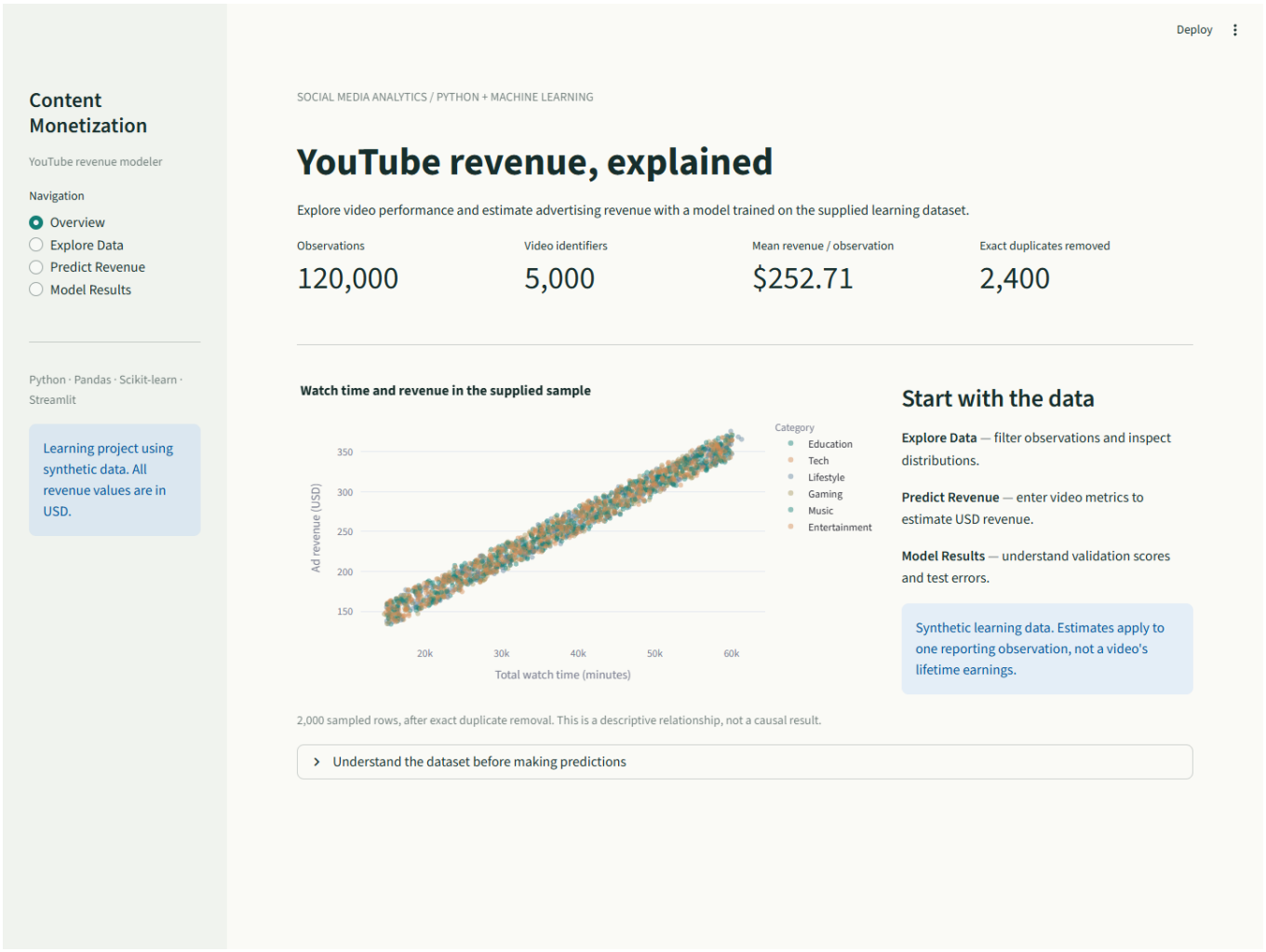
10 feature importance



Permutation importance on validation before refitting shows reliance on watch time.

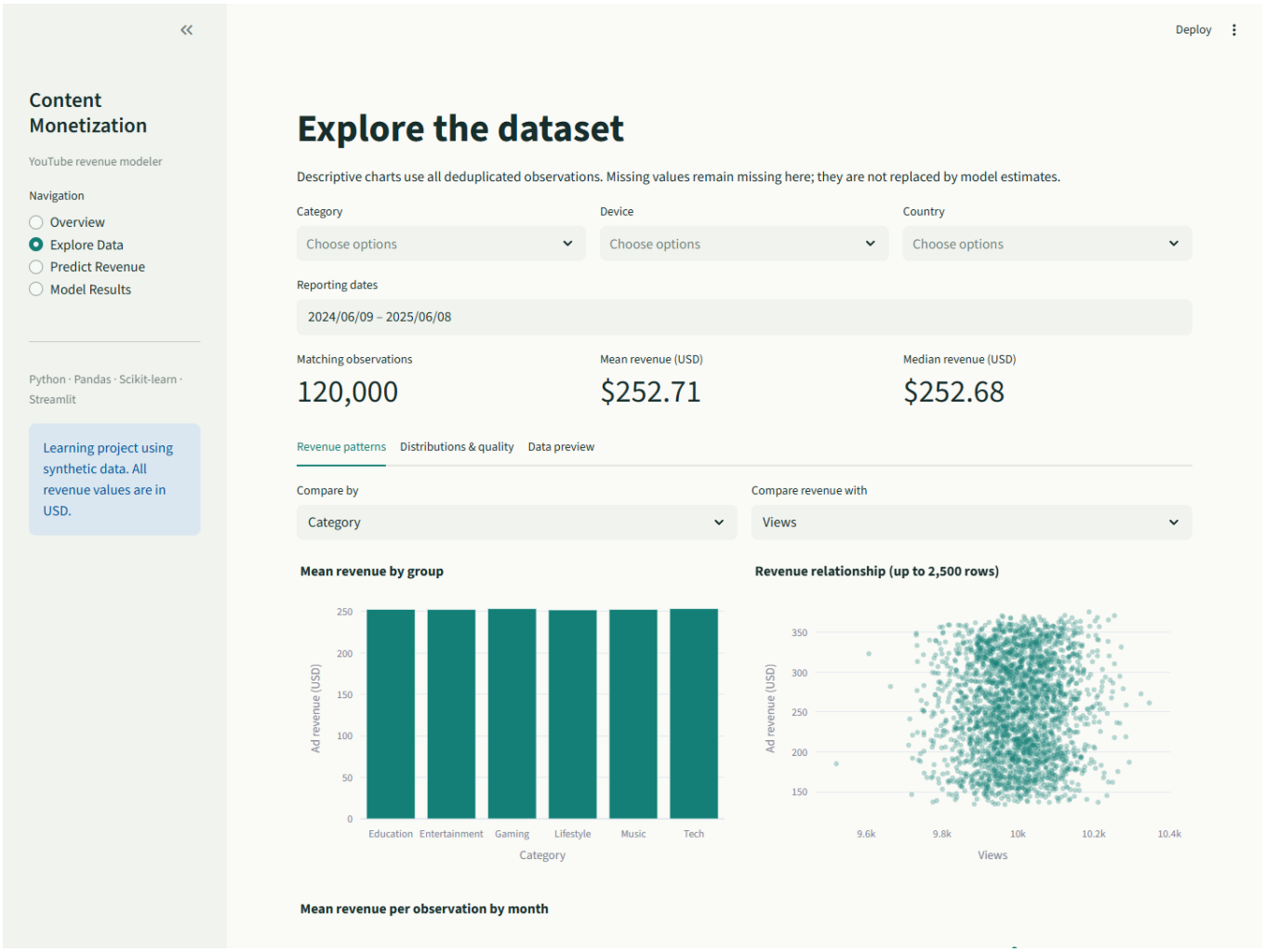
13. Application screenshots

Overview: dataset context, sample statistics and navigation.



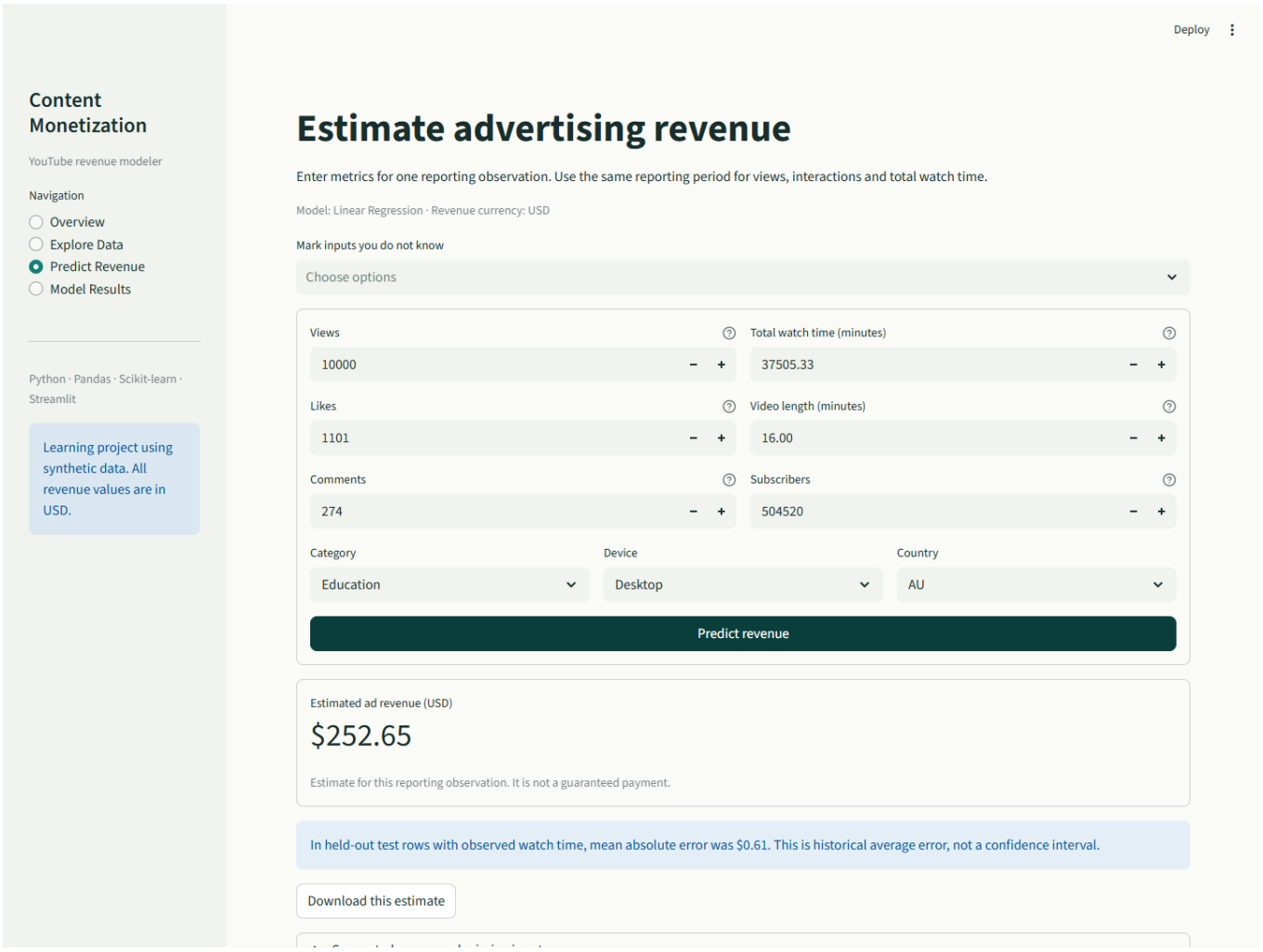
Captured from the running Streamlit application in a desktop browser. Scroll inside the app to see additional content.

Explore Data: interactive filters and descriptive visualizations.



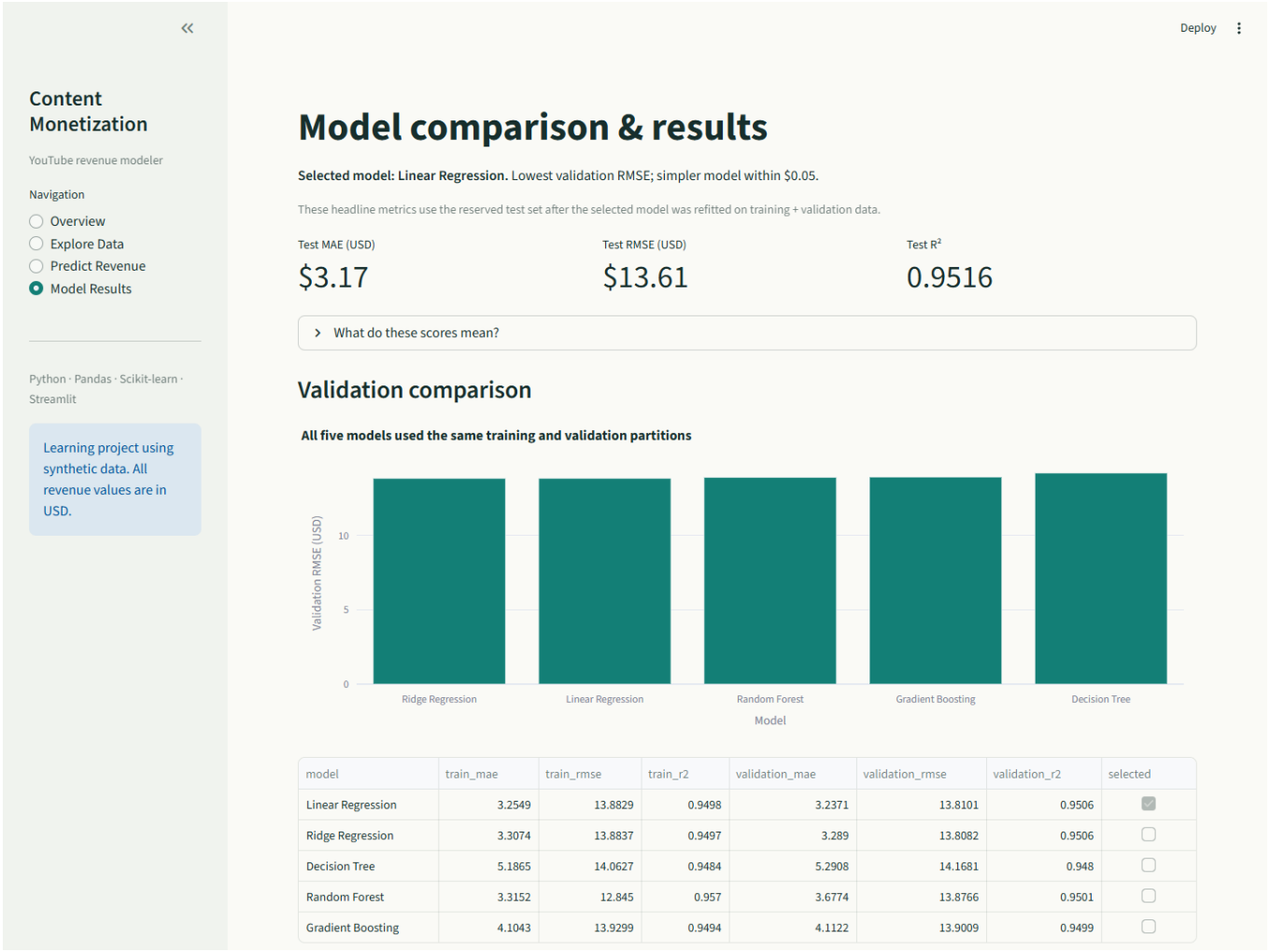
Captured from the running Streamlit application in a desktop browser. Scroll inside the app to see additional content.

Predict Revenue: a completed form and an estimate of approximately $252.65.



Captured from the running Streamlit application in a desktop browser. Scroll inside the app to see additional content.

Model Results: the upper screen, with reserved test metrics and validation comparison.



Captured from the running Streamlit application in a desktop browser. Scroll inside the app to see additional content.

14. Python reading key

Syntax	Meaning in this project
import / from ... import	Make a library or shared function available. as gives it a short alias.
name = value	Assign a value or returned result to a name. A list holds ordered items; a dictionary maps keys to values.
def ... / return	Define a reusable function and send a result back to its caller.
if / elif / else	Choose a branch according to a condition. Indented statements belong to that branch.
for / in / zip	Repeat work over items. zip pairs corresponding items from two sequences.
with	Enter a context, such as a Streamlit column/form or an expected-error test.
try / except / raise	Handle a possible failure, catch it at a useful boundary, or report invalid data.
assert / pytest.approx	Check a requirement; approximate equality permits tiny floating-point differences.
f"...{value:.2f}"	Insert a value into readable text and display two digits after the decimal point.
df[column] / loc / iloc	Select named columns, labeled rows/columns or integer-position rows.
fit / transform / predict	Learn parameters, apply fitted preparation, or estimate revenue.
@decorator	Wrap a function, for example with Streamlit caching or pytest fixtures/parameterization.
__name__ == "__main__"	Run the entry point when a file is executed directly rather than imported.

Some formatting calls or long dictionaries span several source lines. The explanation labels the whole statement. For a condition or loop, its annotation describes the header; the nested statements have their own entries. The complete code is reproduced, so you can compare the guide with the actual file.

15. Code walkthrough: utils.py

Read this file first. It owns the reusable rules that keep training and application predictions consistent.

utils.py | lines 1-24

```
 1  """Small shared functions for data preparation, training and prediction."""
 2
 3  from pathlib import Path
 4  import hashlib
 5  import json
 6
 7  import numpy as np
 8  import pandas as pd
 9  from sklearn.compose import ColumnTransformer
10  from sklearn.impute import SimpleImputer
11  from sklearn.model_selection import GroupShuffleSplit
12  from sklearn.pipeline import Pipeline
13  from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler
14
15  ROOT = Path(__file__).resolve().parent
16  RAW_DATA = ROOT / "data" / "youtube_ad_revenue_dataset.csv"
17  TARGET = "ad_revenue_usd"
18  SEED = 42
19  NUMERIC = ["views", "likes", "comments", "watch_time_minutes",
20             "video_length_minutes", "subscribers"]
21  CATEGORICAL = ["category", "device", "country"]
22  INPUTS = NUMERIC + CATEGORICAL
23  DERIVED = ["engagement_rate", "average_watch_minutes"]
24  OPTIONAL = ["likes", "comments", "watch_time_minutes"]
```

Line 3: Import Path from pathlib, providing portable file paths.

Line 4: Import hashlib for SHA-256 file fingerprints.

Line 5: Import json for readable metadata and report files.

Line 7: Import numpy as np for numeric arrays and finite-value checks.

Line 8: Import pandas as pd for DataFrames and CSV analysis.

Line 9: Import ColumnTransformer from sklearn.compose, providing machine-learning models and preparation.

Line 10: Import SimpleImputer from sklearn.impute, providing machine-learning models and preparation.

Line 11: Import GroupShuffleSplit from sklearn.model_selection, providing machine-learning models and preparation.

Line 12: Import Pipeline from sklearn.pipeline, providing machine-learning models and preparation.

Line 13: Import FunctionTransformer, OneHotEncoder, StandardScaler from sklearn.preprocessing, providing machine-learning models and preparation.

Line 15: Assign ROOT: read the indicated property from the existing object.

Line 16: Assign RAW_DATA: calculate the shown arithmetic or combine the indicated collections.

Line 17: Assign TARGET: store this literal value as a setting or initial value.

Line 18: Assign SEED: store this literal value as a setting or initial value.

Line 19-20: Assign NUMERIC: collect these values in the specified order.

Line 21: Assign CATEGORICAL: collect these values in the specified order.

Line 22: Assign INPUTS: calculate the shown arithmetic or combine the indicated collections.

Line 23: Assign DERIVED: collect these values in the specified order.

Line 24: Assign OPTIONAL: collect these values in the specified order.

utils.py | lines 25-48

file_hash: Read the file bytes and compute a SHA-256 fingerprint. A changed file normally produces a changed fingerprint. This links artifacts to their metadata without modifying the file.

save_json: Serialize a dictionary as readable JSON. allow_nan=False deliberately rejects NaN or infinity so incomplete numeric results cannot silently become invalid report data.

read_json: Read UTF-8 text and turn its JSON objects into Python dictionaries, lists and numbers. Missing or malformed files raise errors for the caller to handle.

load_data: Select the expected columns and validate their meaning. Numeric conversions and ISO dates fail clearly on malformed values. Essential fields cannot be missing, counts must be whole numbers and views/length must be positive. Whitespace-only IDs are rejected. Exact duplicate removal happens after normalization. Return the deduplicated DataFrame and a separate quality dictionary.

```
25
26
27  def file_hash(path):
28      """Identify a file so reports can be matched to their data and model."""
29      return hashlib.sha256(Path(path).read_bytes()).hexdigest()
30
31
32  def save_json(data, path):
33      Path(path).write_text(json.dumps(data, indent=2, allow_nan=False), encoding="utf-8")
34
35
36  def read_json(path):
37      return json.loads(Path(path).read_text(encoding="utf-8"))
38
39
40  def load_data(path=RAW_DATA):
41      """Validate the source and remove exact duplicates without changing the file."""
42      data = pd.read_csv(path)
43      required = ["video_id", "date"] + INPUTS + [TARGET]
44      missing = sorted(set(required) - set(data.columns))
45      if missing:
46          raise ValueError("Missing columns: " + ", ".join(missing))
47      if data.empty:
48          raise ValueError("The dataset has no rows.")
```

Line 27: Define file_hash(path). The indented body runs when this function is called.

Line 29: Return hashlib.sha256(Path(path).read_bytes()).hexdigest() to the caller; this ends the current function call.

Line 32: Define save_json(data, path). The indented body runs when this function is called.

Line 33: Call path(path).write_text with the shown arguments.

Line 36: Define read_json(path). The indented body runs when this function is called.

Line 37: Return json.loads(Path(path).read_text(encoding='utf-8')) to the caller; this ends the current function call.

Line 40: Define load_data(path). The indented body runs when this function is called.

Line 42: Assign data: read a CSV into a Pandas DataFrame.

Line 43: Assign required: calculate the shown arithmetic or combine the indicated collections.

Line 44: Assign missing: put the items in a deterministic order.

Line 45: Run the following indented branch only when missing is true.

Line 46: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 47: Run the following indented branch only when data.empty is true.

Line 48: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

utils.py | lines 49-72

```
49      data = data[required].copy()
50      for column in NUMERIC + [TARGET]:
51          data[column] = pd.to_numeric(data[column], errors="raise")
52          observed = data[column].dropna()
53          if not np.isfinite(observed).all() or (observed < 0).any():
54              raise ValueError(f"{column} must contain finite, non-negative numbers.")
55          if column in ["views", "likes", "comments", "subscribers"]:
56              if (observed % 1 != 0).any():
57                  raise ValueError(f"{column} must contain whole-number counts.")
58      essential = ["video_id", "date", "views", "video_length_minutes", "subscribers", TARGET]
59      if data[essential].isna().any().any():
60          raise ValueError("An essential identifier, date or numeric field is missing.")
61      if (data["views"] <= 0).any() or (data["video_length_minutes"] <= 0).any():
62          raise ValueError("Views and video length must be greater than zero.")
63      data["date"] = pd.to_datetime(data["date"], format="ISO8601", errors="raise")
64      for column in ["video_id"] + CATEGORICAL:
65          data[column] = data[column].str.strip().replace("", np.nan)
66      if data["video_id"].isna().any():
67          raise ValueError("Video IDs cannot be blank.")
68      quality = {"raw_rows": len(data), "duplicates_removed": int(data.duplicated().sum())}
69      data = data.drop_duplicates().reset_index(drop=True)
70      quality["clean_rows"] = len(data)
71      quality["missing_after_dedup"] = data.isna().sum().astype(int).to_dict()
72      quality["unique_video_ids"] = int(data["video_id"].nunique())
```

Line 49: Assign data: make a separate copy so later edits do not alter the source object.

Line 50: Repeat the indented steps for each column from NUMERIC + [TARGET].

Line 51: Assign data[column]: convert the column to numbers and reject malformed values.

Line 52: Assign observed: exclude missing values from this calculation.

Line 53: Run the following indented branch only when not np.isfinite(observed).all() or (observed < 0).any() is true.

Line 54: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 55: Run the following indented branch only when column in ['views', 'likes', 'comments', 'subscribers'] is true.

Line 56: Run the following indented branch only when (observed % 1 != 0).any() is true.

Line 57: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 58: Assign essential: collect these values in the specified order.

Line 59: Run the following indented branch only when data[essential].isna().any().any() is true.

Line 60: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 61: Run the following indented branch only when (data['views'] <= 0).any() or (data['video_length_minutes'] <= 0).any() is true.

Line 62: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 63: Assign data['date']: parse the reporting timestamps using the stated date format.

Line 64: Repeat the indented steps for each column from ['video_id'] + CATEGORICAL.

Line 65: Assign data[column]: call data[column].str.strip().replace with the shown arguments.

Line 66: Run the following indented branch only when data['video_id'].isna().any() is true.

Line 67: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 68: Assign quality: build a dictionary mapping these names to their corresponding values.

Line 69: Assign data: restore a simple row index.

Line 70: Assign quality['clean_rows']: count the items or rows.

Line 71: Assign quality['missing_after_dedup']: convert the values into a dictionary for metadata or reporting.

Line 72: Assign quality['unique_video_ids']: convert to an ordinary whole-number integer.

utils.py | lines 73-96

split_data: The first GroupShuffleSplit assigns 60% of distinct IDs to training. A second split divides the remaining 40% evenly into validation and test. next takes the one requested split from its iterator. iloc selects rows by integer position, and copy avoids changing the original DataFrame through a view.

add_features: Select only the nine intended raw inputs. Replacing zero views with NaN makes division safe. Each ratio is calculated from the original observed inputs. Missing ratios are left for the fitted imputer, and the revenue target never enters this result.

```
73      quality["repeated_video_date_rows"] = int(data.duplicated(["video_id", "date"]).sum())
```

```
    return data, quality


def split_data(data):
    """Allocate 60/20/20 percent of video IDs to train/validation/test."""
    if data["video_id"].nunique() < 10:
        raise ValueError("At least 10 distinct video IDs are needed for this split.")
    splitter = GroupShuffleSplit(n_splits=1, train_size=0.6, random_state=SEED)
    train_index, other_index = next(splitter.split(data, groups=data["video_id"]))
    train = data.iloc[train_index].copy()
    other = data.iloc[other_index].copy()
    splitter = GroupShuffleSplit(n_splits=1, test_size=0.5, random_state=SEED)
    val_index, test_index = next(splitter.split(other, groups=other["video_id"]))
    return train, other.iloc[val_index].copy(), other.iloc[test_index].copy()


def add_features(data):
    """Create two ratios; unknown inputs remain unknown until imputation."""
    result = data[INPUTS].copy()
    safe_views = result["views"].replace(0, np.nan)
    result["engagement_rate"] = (result["likes"] + result["comments"]) / safe_views
    result["average_watch_minutes"] = result["watch_time_minutes"] / safe_views
    return result
```

Line 73: Assign quality['repeated_video_date_rows']: convert to an ordinary whole-number integer.

Line 74: Return (data, quality) to the caller; this ends the current function call.

Line 77: Define split_data(data). The indented body runs when this function is called.

Line 79: Run the following indented branch only when data['video_id'].nunique() < 10 is true.

Line 80: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 81: Assign splitter: call GroupShuffleSplit with the shown arguments.

Line 82: Assign (train_index, other_index): take the first and only requested split from the iterator.

Line 83: Assign train: make a separate copy so later edits do not alter the source object.

Line 84: Assign other: make a separate copy so later edits do not alter the source object.

Line 85: Assign splitter: call GroupShuffleSplit with the shown arguments.

Line 86: Assign (val_index, test_index): take the first and only requested split from the iterator.

Line 87: Return (train, other.iloc[val_index].copy(), other.iloc[test_index].copy()) to the caller; this ends the current function call.

Line 90: Define add_features(data). The indented body runs when this function is called.

Line 92: Assign result: make a separate copy so later edits do not alter the source object.

Line 93: Assign safe_views: call result['views'].replace with the shown arguments.

Line 94: Assign result['engagement_rate']: calculate the shown arithmetic or combine the indicated collections.

Line 95: Assign result['average_watch_minutes']: calculate the shown arithmetic or combine the indicated collections.

Line 96: Return result to the caller; this ends the current function call.

utils.py | lines 97-120

build_pipeline: Connect three major stages: feature creation, column-specific preparation and estimation. The numeric branch adds missing indicators and scales its output. The categorical branch imputes text and creates one-hot indicators. FunctionTransformer wraps our normal top-level function without inventing a custom transformer class. Its importable name must remain in utils.py so the saved pipeline can load.

make_input_metadata: Build dictionaries for the form's supported ranges, typical defaults and category choices from final development rows. Continuous limits are rounded inward so the two-decimal UI endpoints stay within observed support. These dictionaries are saved with model details, not learned from the test set.

```


def build_pipeline(estimator):
    """Save feature creation, imputation, encoding, scaling and the model together."""
    numbers = Pipeline([
        ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
        ("scaler", StandardScaler()),
    ])
    categories = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ])
```

```
109      preparation = ColumnTransformer([
110          ("numbers", numbers, NUMERIC + DERIVED),
111          ("categories", categories, CATEGORICAL),
112      ])
113      return Pipeline([
114          ("features", FunctionTransformer(add_features, validate=False)),
115          ("preparation", preparation),
116          ("model", estimator),
117      ])
118
119
120  def make_input_metadata(data):
```

Line 99: Define build_pipeline(estimator). The indented body runs when this function is called.

Line 101-104: Assign numbers: call Pipeline with the shown arguments.

Line 105-108: Assign categories: call Pipeline with the shown arguments.

Line 109-112: Assign preparation: call ColumnTransformer with the shown arguments.

Line 113-117: Return Pipeline([('features', FunctionTransformer(add_features, validate=False)), ('preparation', prepar... to the caller; this ends the current function call.

Line 120: Define make_input_metadata(data). The indented body runs when this function is called.

utils.py | lines 121-144

predict_revenue: Build exactly one named row from a user's values. Optional unknown inputs become NaN. Required fields, finite numbers, non-negative values, integer counts and known categories are checked. Values outside the fitted ranges are refused. A watch-time/length inconsistency produces a note. A negative or non-finite prediction raises an error rather than being silently clipped. The result is a float and a list of explanatory notes.

```
121      """Describe only the data used to fit the final model, not the test data."""
122      metadata = {"ranges": {}, "defaults": {}, "categories": {}}
123      for column in NUMERIC:
124          low, high = float(data[column].min()), float(data[column].max())
125          # Round inward so the two-decimal limits shown in the app are actually valid.
126          if column in ["watch_time_minutes", "video_length_minutes"]:
127              low, high = float(np.ceil(low * 100) / 100), float(np.floor(high * 100) / 100)
128          metadata["ranges"][column] = [low, high]
129          metadata["defaults"][column] = float(data[column].median())
130      for column in CATEGORICAL:
131          metadata["categories"][column] = sorted(data[column].dropna().unique().tolist())
132      return metadata
133
134
135  def predict_revenue(pipeline, values, metadata):
136      """Validate one input. Refuse unsupported ranges instead of hiding extrapolation."""
137      row = {}
138      notes = []
139      for column in NUMERIC:
140          value = values.get(column)
141          if value is None or pd.isna(value):
142              if column not in OPTIONAL:
143                  raise ValueError(f"Please provide {column.replace('_', ' ')}.")
144              row[column] = np.nan
```

Line 122: Assign metadata: build a dictionary mapping these names to their corresponding values.

Line 123: Repeat the indented steps for each column from NUMERIC.

Line 124: Assign (low, high): collect these values in the specified order.

Line 126: Run the following indented branch only when column in ['watch_time_minutes', 'video_length_minutes'] is true.

Line 127: Assign (low, high): collect these values in the specified order.

Line 128: Assign metadata['ranges'][column]: collect these values in the specified order.

Line 129: Assign metadata['defaults'][column]: convert to an ordinary Python floating-point number.

Line 130: Repeat the indented steps for each column from CATEGORICAL.

Line 131: Assign metadata['categories'][column]: put the items in a deterministic order.

Line 132: Return metadata to the caller; this ends the current function call.

Line 135: Define predict_revenue(pipeline, values, metadata). The indented body runs when this function is called.

Line 137: Assign row: build a dictionary mapping these names to their corresponding values.

Line 138: Assign notes: collect these values in the specified order.

Line 139: Repeat the indented steps for each column from NUMERIC.

Line 140: Assign value: look up a key, returning its default if the key is absent.

Line 141: Run the following indented branch only when value is None or pd.isna(value) is true.

Line 142: Run the following indented branch only when column not in OPTIONAL is true.

Line 143: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 144: Assign row[column]: read the indicated property from the existing object.

utils.py | lines 145-168

```
145             notes.append(f"{column.replace('_', ' ')} is unknown; saved medians are used.")
146             continue
147         try:
148             value = float(value)
149         except (TypeError, ValueError) as error:
150             raise ValueError(f"{column} must be a number.") from error
151         if not np.isfinite(value) or value < 0:
152             raise ValueError(f"{column} must be finite and non-negative.")
153         if column in ["views", "likes", "comments", "subscribers"] and not value.is_integer():
154             raise ValueError(f"{column} must be a whole number.")
155         low, high = metadata["ranges"][column]
156         if not low <= value <= high:
157             raise ValueError(f"{column.replace('_', ' ')} is outside the fitted range "
158                              f"({low:,.2f} to {high:,.2f}). Use a value in this range.")
159         row[column] = value
160     for column in CATEGORICAL:
161         value = values.get(column)
162         if value not in metadata["categories"][column]:
163             raise ValueError(f"Choose a supported {column}.")
164         row[column] = value
165     if row["watch_time_minutes"] / row["views"] > row["video_length_minutes"]:
166         notes.append("Average watch time exceeds video length. Check the reporting "
167                      "definitions; similar observations occur in the synthetic data.")
168     prediction = float(pipeline.predict(pd.DataFrame([row]))[0])
```

Line 145: Append this result to the accumulating list.

Line 146: Move to the next loop item. This avoids applying later numeric checks to an input already marked unknown.

Line 147: Try the following operation; a matching exception is handled by the following recovery branch.

Line 148: Assign value: convert to an ordinary Python floating-point number.

Line 149: Catch the named exception and enter the recovery or expected-error branch.

Line 150: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 151: Run the following indented branch only when not np.isfinite(value) or value < 0 is true.

Line 152: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 153: Run the following indented branch only when column in ['views', 'likes', 'comments', 'subscribers'] and (not value.is_integer()) is true.

Line 154: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 155: Assign (low, high): select the named field, columns or rows from the existing object.

Line 156: Run the following indented branch only when not low <= value <= high is true.

Line 157-158: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 159: Assign row[column]: reuse the existing value named value.

Line 160: Repeat the indented steps for each column from CATEGORICAL.

Line 161: Assign value: look up a key, returning its default if the key is absent.

Line 162: Run the following indented branch only when value not in metadata['categories'][column] is true.

Line 163: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 164: Assign row[column]: reuse the existing value named value.

Line 165: Run the following indented branch only when row['watch_time_minutes'] / row['views'] > row['video_length_minutes'] is true.

Line 166-167: Append this result to the accumulating list.

Line 168: Assign prediction: convert to an ordinary Python floating-point number.

utils.py | lines 169-171

```
169     if not np.isfinite(prediction) or prediction < 0:
170         raise ValueError("The model could not return a valid non-negative estimate.")
171     return prediction, notes
```

Line 169: Run the following indented branch only when not np.isfinite(prediction) or prediction < 0 is true.

Line 170: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 171: Return (prediction, notes) to the caller; this ends the current function call.

16. Code walkthrough: train_model.py

This is the training entry point. It calls the shared utilities and creates all generated model/report artifacts.

train_model.py | lines 1-24

regression_metrics: Calculate MAE, RMSE and R^2 from actual and predicted arrays. Convert NumPy-like outputs to ordinary Python floats so JSON serialization is predictable.

```
1  """Run this file to train all five models and regenerate the model and reports."""
2
3  import platform
4  import time
5  from importlib.metadata import version
6
7  import joblib
8  import numpy as np
9  import pandas as pd
10 from sklearn.base import clone
11 from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
12 from sklearn.inspection import permutation_importance
13 from sklearn.linear_model import LinearRegression, Ridge
14 from sklearn.metrics import mean_absolute_error, r2_score, root_mean_squared_error
15 from sklearn.tree import DecisionTreeRegressor
16
17 from charts import create_figures
18 from utils import (ROOT, RAW_DATA, TARGET, SEED, INPUTS, NUMERIC, CATEGORICAL, DERIVED,
19                    add_features, build_pipeline, file_hash, load_data,
20                    make_input_metadata, save_json, split_data)
21
22
23 def regression_metrics(actual, predicted):
24     return {"mae": float(mean_absolute_error(actual, predicted)),
```

Line 3: Import platform for the Python version recorded in metadata.

Line 4: Import time for elapsed-time measurement.

Line 5: Import version from importlib.metadata, providing installed package version information.

Line 7: Import joblib for saving and loading fitted Python models.

Line 8: Import numpy as np for numeric arrays and finite-value checks.

Line 9: Import pandas as pd for DataFrames and CSV analysis.

Line 10: Import clone from sklearn.base, providing machine-learning models and preparation.

Line 11: Import GradientBoostingRegressor, RandomForestRegressor from sklearn.ensemble, providing machine-learning models and preparation.

Line 12: Import permutation_importance from sklearn.inspection, providing machine-learning models and preparation.

Line 13: Import LinearRegression, Ridge from sklearn.linear_model, providing machine-learning models and preparation.

Line 14: Import mean_absolute_error, r2_score, root_mean_squared_error from sklearn.metrics, providing machine-learning models and preparation.

Line 15: Import DecisionTreeRegressor from sklearn.tree, providing machine-learning models and preparation.

Line 17: Import create_figures from charts, providing saved figure generation.

Line 18-20: Import ROOT, RAW_DATA, TARGET, SEED, INPUTS, NUMERIC, CATEGORICAL, DERIVED, add_features, build_pipeline, file_hash, load_data, make_input_metadata, save_json, split_data from utils, providing shared project rules.

Line 23: Define regression_metrics(actual, predicted). The indented body runs when this function is called.

Line 24-26: Return {'mae': float(mean_absolute_error(actual, predicted)), 'rmse': float(root_mean_squared_error(actu... to the caller; this ends the current function call.

train_model.py | lines 25-48

get_models: A dictionary gives each candidate a readable name and a scikit-learn estimator. Settings limit tree complexity and fix randomness. Dictionary insertion order is the stated simplicity order used only when validation RMSE values are within the predefined tolerance.

export_clean_data: Create an unscaled analysis export. Use the fitted numeric imputer before its scaler, take the first eight transformed columns as raw/derived numeric values, and write explicit flags for original missingness. Use the fitted categorical imputer and attach the ID-to-partition mapping. Preserve the original source CSV.

```
25              "rmse": float(root_mean_squared_error(actual, predicted)),
26              "r2": float(r2_score(actual, predicted))}
27
28
29  def get_models():
30      """Fixed, modest settings keep the comparison reproducible and easy to explain."""
31      return {
32          "Linear Regression": LinearRegression(),
33          "Ridge Regression": Ridge(alpha=10.0),
34          "Decision Tree": DecisionTreeRegressor(max_depth=8, min_samples_leaf=20,
35                                                 random_state=SEED),
36          "Random Forest": RandomForestRegressor(n_estimators=80, max_depth=12,
37                                                 min_samples_leaf=8, n_jobs=2,
38                                                 random_state=SEED),
39          "Gradient Boosting": GradientBoostingRegressor(n_estimators=100, max_depth=2,
40                                                         min_samples_leaf=20,
41                                                         random_state=SEED),
42      }
43
44
45  def export_clean_data(data, pipeline, partitions):
46      """Export unscaled, readable values using medians learned from train + validation."""
47      cleaned = data.copy()
48      features = add_features(data)
```

Line 29: Define get_models(). The indented body runs when this function is called.

Line 31-42: Return {'Linear Regression': LinearRegression(), 'Ridge Regression': Ridge(alpha=10.0), 'Decision Tree':... to the caller; this ends the current function call.

Line 45: Define export_clean_data(data, pipeline, partitions). The indented body runs when this function is called.

Line 47: Assign cleaned: make a separate copy so later edits do not alter the source object.

Line 48: Assign features: select the approved inputs and calculate the two ratio features.

train_model.py | lines 49-72

run_training: Create output directories, load validated data and separate IDs. For every candidate fit the complete pipeline on training, score train/validation and collect a comparison table. Apply the predefined tolerance, measure validation permutation importance before refitting, then fit a fresh selected model on development rows. Score the reserved test set and its watch-time groups. Save predictions, partitions, quality, the model and metadata. Check serialization immediately by comparing predictions from the reloaded model. Finally export cleaned data and the ten figures.

```
49      number_imputer = pipeline["preparation"].named_transformers_["numbers"]["imputer"]
50      filled = number_imputer.transform(features[NUMERIC + DERIVED])
51      cleaned[NUMERIC + DERIVED] = filled[:, :len(NUMERIC + DERIVED)]
52      for column in NUMERIC + DERIVED:
53          cleaned[column + "_was_missing"] = features[column].isna()
54      category_imputer = pipeline["preparation"].named_transformers_["categories"]["imputer"]
55      cleaned[CATEGORICAL] = category_imputer.transform(features[CATEGORICAL])
56      cleaned["partition"] = cleaned["video_id"].map(partitions)
57      cleaned.to_csv(ROOT / "data" / "cleaned_youtube_data.csv", index=False)
58
59
60  def run_training():
61      """Compare on validation, refit the selected model, then evaluate test data once."""
62      for name in ["models", "reports", "reports/figures"]:
63          (ROOT / name).mkdir(parents=True, exist_ok=True)
64      data, quality = load_data()
65      train, validation, test = split_data(data)
66      models = get_models()
67      fitted = {}
68      rows = []
69      for name, estimator in models.items():
70          print(f"Training {name} ...", flush=True)
71          started = time.perf_counter()
72          pipeline = build_pipeline(estimator)
```

Line 49: Assign number_imputer: select the named field, columns or rows from the existing object.

Line 50: Assign filled: apply an already-fitted transformation without relearning its parameters.

Line 51: Assign cleaned[NUMERIC + DERIVED]: select the named field, columns or rows from the existing object.

Line 52: Repeat the indented steps for each column from NUMERIC + DERIVED.

Line 53: Assign cleaned[column + '_was_missing']: identify missing values with a Boolean mask.

Line 54: Assign category_imputer: select the named field, columns or rows from the existing object.

Line 55: Assign cleaned[CATEGORICAL]: apply an already-fitted transformation without relearning its parameters.

Line 56: Assign cleaned['partition']: call cleaned['video_id'].map with the shown arguments.

Line 57: Write the table to CSV without adding a redundant row-index column.

Line 60: Define run_training(). The indented body runs when this function is called.

Line 62: Repeat the indented steps for each name from ['models', 'reports', 'reports/figures'].

Line 63: Create the output folder if it does not already exist.

Line 64: Assign (data, quality): load, validate and deduplicate the source, returning data and quality counts.

Line 65: Assign (train, validation, test): separate observations into training, validation and test by video ID.

Line 66: Assign models: create the five candidate estimators in the predefined simplicity order.

Line 67: Assign fitted: build a dictionary mapping these names to their corresponding values.

Line 68: Assign rows: collect these values in the specified order.

Line 69: Repeat the indented steps for each (name, estimator) from models.items().

Line 70: Write the status or result to the terminal or notebook output.

Line 71: Assign started: read a precise elapsed-time clock.

Line 72: Assign pipeline: construct a fresh pipeline around the supplied estimator.

train_model.py | lines 73-96

```
73          pipeline.fit(train[INPUTS], train[TARGET])
74          row = {"model": name}
75          for label, frame in [("train", train), ("validation", validation)]:
76              scores = regression_metrics(frame[TARGET], pipeline.predict(frame[INPUTS]))
77              row.update({label + "_" + key: value for key, value in scores.items()})
78          row["fit_and_score_seconds"] = round(time.perf_counter() - started, 2)
79          rows.append(row)
80          fitted[name] = pipeline
81      comparison = pd.DataFrame(rows)
82      # Within five cents of the lowest RMSE, prefer the earlier, simpler model.
83      limit = comparison["validation_rmse"].min() + 0.05
84      chosen = comparison.loc[comparison["validation_rmse"] <= limit, "model"].iloc[0]
85      comparison["selected"] = comparison["model"] == chosen
86      comparison.to_csv(ROOT / "reports" / "model_comparison.csv", index=False)
87
88      # Measure original input importance on validation BEFORE refitting on it.
89      sample = validation.sample(min(1500, len(validation)), random_state=SEED)
90      result = permutation_importance(fitted[chosen], sample[INPUTS], sample[TARGET],
91                                      scoring="neg_mean_absolute_error", n_repeats=3,
92                                      random_state=SEED, n_jobs=1)
93      importance = pd.DataFrame({"feature": INPUTS, "mae_increase_usd": result.importances_mean,
94                                 "std_usd": result.importances_std})
95      importance.to_csv(ROOT / "reports" / "feature_importance.csv", index=False)
96
```

Line 73: Learn preparation and model parameters from the explicitly supplied fitting rows.

Line 74: Assign row: build a dictionary mapping these names to their corresponding values.

Line 75: Repeat the indented steps for each (label, frame) from [('train', train), ('validation', validation)].

Line 76: Assign scores: calculate MAE, RMSE and R^2 for the supplied actual and predicted values.

Line 77: Merge these new fields into the existing dictionary.

Line 78: Assign row['fit_and_score_seconds']: round a value for display or timing, without changing the underlying model.

Line 79: Append this result to the accumulating list.

Line 80: Assign fitted[name]: reuse the existing value named pipeline.

Line 81: Assign comparison: construct a named two-dimensional table.

Line 83: Assign limit: calculate the shown arithmetic or combine the indicated collections.

Line 84: Assign chosen: select the named field, columns or rows from the existing object.

Line 85: Assign comparison['selected']: evaluate the comparison, producing a Boolean value or mask.

Line 86: Write the table to CSV without adding a redundant row-index column.

Line 89: Assign sample: choose a reproducible subset for display or importance analysis.

Line 90-92: Assign result: shuffle each original input and measure its validation-score effect.

Line 93-94: Assign importance: construct a named two-dimensional table.

Line 95: Write the table to CSV without adding a redundant row-index column.

train_model.py | lines 97-120

```
 97      development = pd.concat([train, validation])
 98      final_pipeline = build_pipeline(clone(models[chosen]))
 99      final_pipeline.fit(development[INPUTS], development[TARGET])
100      predicted = final_pipeline.predict(test[INPUTS])
101      scores = regression_metrics(test[TARGET], predicted)
102      baseline = np.full(len(test), development[TARGET].mean())
103      scores["mean_baseline"] = regression_metrics(test[TARGET], baseline)
104      scores["model"] = chosen
105      scores["test_rows"] = len(test)
106      scores["watch_time_groups"] = {}
107      for missing in [False, True]:
108          mask = test["watch_time_minutes"].isna().to_numpy() == missing
109          label = "missing" if missing else "observed"
110          group_scores = regression_metrics(test.loc[mask, TARGET], predicted[mask])
111          scores["watch_time_groups"][label] = {"rows": int(mask.sum()), **group_scores}
112      save_json(scores, ROOT / "reports" / "test_metrics.json")
113
114      predictions = pd.DataFrame({"video_id": test["video_id"], "actual": test[TARGET],
115                                  "predicted": predicted,
116                                  "watch_time_missing": test["watch_time_minutes"].isna()})
117      predictions.to_csv(ROOT / "reports" / "test_predictions.csv", index=False)
118      partitions = {}
119      split_summary = {}
120      for label, frame in [("train", train), ("validation", validation), ("test", test)]:
```

Line 97: Assign development: join the supplied DataFrames; here this forms development data.

Line 98: Assign final_pipeline: construct a fresh pipeline around the supplied estimator.

Line 99: Learn preparation and model parameters from the explicitly supplied fitting rows.

Line 100: Assign predicted: apply the fitted pipeline to obtain revenue estimates.

Line 101: Assign scores: calculate MAE, RMSE and R^2 for the supplied actual and predicted values.

Line 102: Assign baseline: create an array filled with the development target mean for the simple baseline.

Line 103: Assign scores['mean_baseline']: calculate MAE, RMSE and R^2 for the supplied actual and predicted values.

Line 104: Assign scores['model']: reuse the existing value named chosen.

Line 105: Assign scores['test_rows']: count the items or rows.

Line 106: Assign scores['watch_time_groups']: build a dictionary mapping these names to their corresponding values.

Line 107: Repeat the indented steps for each missing from [False, True].

Line 108: Assign mask: evaluate the comparison, producing a Boolean value or mask.

Line 109: Assign label: choose the first value when the condition is true, otherwise the alternative.

Line 110: Assign group_scores: calculate MAE, RMSE and R^2 for the supplied actual and predicted values.

Line 111: Assign scores['watch_time_groups'][label]: build a dictionary mapping these names to their corresponding values.

Line 112: Write the dictionary as readable, finite-valued JSON.

Line 114-116: Assign predictions: construct a named two-dimensional table.

Line 117: Write the table to CSV without adding a redundant row-index column.

Line 118: Assign partitions: build a dictionary mapping these names to their corresponding values.

Line 119: Assign split_summary: build a dictionary mapping these names to their corresponding values.

Line 120: Repeat the indented steps for each (label, frame) from [('train', train), ('validation', validation), ('test', test)].

train_model.py | lines 121-144

```
121         partitions.update(dict.fromkeys(frame["video_id"].unique(), label))
122         split_summary[label] = {"rows": len(frame), "video_ids": int(frame.video_id.nunique())}
123     pd.DataFrame(list(partitions.items()), columns=["video_id", "partition"]).to_csv(
124         ROOT / "reports" / "data_partitions.csv", index=False)
125     save_json(quality, ROOT / "reports" / "data_quality.json")
126     model_path = ROOT / "models" / "revenue_pipeline.joblib"
127     joblib.dump(final_pipeline, model_path, compress=3)
128     # A round-trip check catches serialization or feature-order mistakes immediately.
129     restored = joblib.load(model_path)
130     np.testing.assert_allclose(restored.predict(test[INPUTS].head(20)), predicted[:20])
131     metadata = make_input_metadata(development)
132     metadata.update({"model": chosen, "seed": SEED, "split": split_summary,
133                      "model_sha256": file_hash(model_path), "data_sha256": file_hash(RAW_DATA),
134                      "python_version": platform.python_version(),
135                      "versions": {name: version(name) for name in
136                                   ["scikit-learn", "numpy", "pandas", "joblib"]},
137                      "selection_rule": "Lowest validation RMSE; simpler model within $0.05",
138                      "fitted_on": "Training and validation IDs; test IDs excluded"})
139     save_json(metadata, ROOT / "models" / "metadata.json")
140     export_clean_data(data, final_pipeline, partitions)
141     create_figures(train, quality, comparison, predictions, importance)
142     print(f"Selected {chosen}. Test MAE=${scores['mae']:.2f}, "
143           f"RMSE=${scores['rmse']:.2f}, R2={scores['r2']:.4f}", flush=True)
144     return comparison, scores
```

Line 121: Merge these new fields into the existing dictionary.

Line 122: Assign split_summary[label]: build a dictionary mapping these names to their corresponding values.

Line 123-124: Write the table to CSV without adding a redundant row-index column.

Line 125: Write the dictionary as readable, finite-valued JSON.

Line 126: Assign model_path: calculate the shown arithmetic or combine the indicated collections.

Line 127: Serialize the fitted pipeline to the model file.

Line 129: Assign restored: restore the fitted pipeline from the trusted saved artifact.

Line 130: Verify numeric agreement within floating-point tolerance.

Line 131: Assign metadata: derive form ranges, defaults and category choices from fitted development data.

Line 132-138: Merge these new fields into the existing dictionary.

Line 139: Write the dictionary as readable, finite-valued JSON.

Line 140: Call export_clean_data with the shown arguments.

Line 141: Call create_figures with the shown arguments.

Line 142-143: Write the status or result to the terminal or notebook output.

Line 144: Return (comparison, scores) to the caller; this ends the current function call.

train_model.py | lines 145-148

```
145
146
147 if __name__ == "__main__":
148     run_training()
```

Line 147: Run this entry point only when the file is executed directly, rather than imported.

Line 148: Call run_training with the shown arguments.

17. Code walkthrough: charts.py

This file makes static, exportable analysis figures. It is separate so chart construction does not obscure the training logic.

charts.py | lines 1-24

create_figures: Use training rows for descriptive model-development plots, all deduplicated rows for the missingness audit, validation for comparison/importance and reserved test rows for error plots. Labels explicitly distinguish these populations. Scatter plots use reproducible samples for legibility. Each figure is closed after saving so repeated training does not accumulate open figures.

finish: This nested helper has access to the outer folder and paths list. It tightens the figure layout, writes a PNG, closes the figure and remembers its path. It is used only while creating charts and is not part of the serialized prediction pipeline.

```
"""Create readable figures for the notebook, reports and explanation PDF."""

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

from utils import ROOT, NUMERIC, TARGET


def create_figures(train, quality, comparison, predictions, importance):
    """EDA uses training rows. Error plots use the final, reserved test rows."""
    folder = ROOT / "reports" / "figures"
    folder.mkdir(parents=True, exist_ok=True)
    sns.set_theme(style="whitegrid", palette="deep", font_scale=1.05)
    paths = []

    def finish(figure, name):
        figure.tight_layout()
        path = folder / f"{name}.png"
        figure.savefig(path, dpi=160, bbox_inches="tight")
        plt.close(figure)
        paths.append(path)

```

Line 3: Import matplotlib for static exportable figures.

Line 4: Choose a non-interactive plotting backend for scripts and headless verification.

Line 5: Import matplotlib.pyplot as plt for static exportable figures.

Line 6: Import seaborn as sns for statistical charts.

Line 8: Import ROOT, NUMERIC, TARGET from utils, providing shared project rules.

Line 11: Define create_figures(train, quality, comparison, predictions, importance). The indented body runs when this function is called.

Line 13: Assign folder: calculate the shown arithmetic or combine the indicated collections.

Line 14: Create the output folder if it does not already exist.

Line 15: Set consistent statistical-chart styling.

Line 16: Assign paths: collect these values in the specified order.

Line 18: Define finish(figure, name). The indented body runs when this function is called.

Line 19: Adjust figure spacing so labels fit.

Line 20: Assign path: calculate the shown arithmetic or combine the indicated collections.

Line 21: Export the figure as a sharp PNG with a tight bounding box.

Line 22: Close the figure to release plotting resources.

Line 23: Append this result to the accumulating list.

charts.py | lines 25-48

```
25      figure, axis = plt.subplots(figsize=(9, 4))
26      missing = quality["missing_after_dedup"]
27      missing = {key: value for key, value in missing.items() if value > 0}
28      axis.bar(list(missing), list(missing.values()), color="#18857d")
29      axis.set(title="Missing values after duplicate removal (all rows)", ylabel="Rows")
30      finish(figure, "01_missing_values")
31
32      figure, axes = plt.subplots(2, 2, figsize=(11, 7))
33      for axis, column in zip(axes.flat, [TARGET, "views", "watch_time_minutes", "likes"]):
34          axis.hist(train[column].dropna(), bins=35, color="#18857d", edgecolor="white")
35          axis.set(title=column.replace("_", " ").title(), ylabel="Training rows")
36      finish(figure, "02_distributions")
37
38      figure, axes = plt.subplots(1, 3, figsize=(12, 4))
39      for axis, column in zip(axes, ["views", "watch_time_minutes", TARGET]):
40          sns.boxplot(y=train[column], ax=axis, color="#86c8ba")
41          axis.set(title="Training distribution", ylabel=column.replace("_", " "))
42      finish(figure, "03_outliers")
43
44      sample = train.sample(min(2500, len(train)), random_state=42)
45      figure, axes = plt.subplots(1, 2, figsize=(12, 4))
46      for axis, column in zip(axes, ["views", "watch_time_minutes"]):
47          axis.scatter(sample[column], sample[TARGET], s=9, alpha=0.25, color="#18857d")
48          axis.set(xlabel=column.replace("_", " "), ylabel="Ad revenue (USD)",
```

Line 25: Assign (figure, axis): create a Matplotlib figure and its axes.

Line 26: Assign missing: select the named field, columns or rows from the existing object.

Line 27: Assign missing: build a dictionary from the shown key/value rule.

Line 28: Draw a vertical bar chart.

Line 29: Set the axis title, labels or other listed properties.

Line 30: Save and close this figure using the local helper.

Line 32: Assign (figure, axes): create a Matplotlib figure and its axes.

Line 33: Repeat the indented steps for each (axis, column) from zip(axes.flat, [TARGET, 'views', 'watch_time_minutes', 'likes']).

Line 34: Plot a numeric distribution using the specified bins.

Line 35: Set the axis title, labels or other listed properties.

Line 36: Save and close this figure using the local helper.

Line 38: Assign (figure, axes): create a Matplotlib figure and its axes.

Line 39: Repeat the indented steps for each (axis, column) from zip(axes, ['views', 'watch_time_minutes', TARGET]).

Line 40: Show the median, quartiles and unusual points of a numeric distribution.

Line 41: Set the axis title, labels or other listed properties.

Line 42: Save and close this figure using the local helper.

Line 44: Assign sample: choose a reproducible subset for display or importance analysis.

Line 45: Assign (figure, axes): create a Matplotlib figure and its axes.

Line 46: Repeat the indented steps for each (axis, column) from zip(axes, ['views', 'watch_time_minutes']).

Line 47: Plot paired observations; transparency helps reveal dense regions.

Line 48-49: Set the axis title, labels or other listed properties.

charts.py | lines 49-72

```
49                  title="2,500 sampled training observations")
50      finish(figure, "04_revenue_relationships")
51
52      figure, axes = plt.subplots(1, 3, figsize=(14, 4))
53      for axis, column in zip(axes, ["category", "country", "device"]):
54          means = train.groupby(column)[TARGET].mean().sort_values()
55          axis.barh(means.index, means.values, color="#18857d")
56          axis.set(xlabel="Mean ad revenue (USD)", title=f"Training sample by {column}")
57          axis.set_xlim(left=0)
58      finish(figure, "05_group_comparisons")
59
60      figure, axis = plt.subplots(figsize=(9, 7))
61      sns.heatmap(train[NUMERIC + [TARGET]].corr(), annot=True, fmt=".2f",
62                  cmap="BrBG", center=0, vmin=-1, vmax=1, ax=axis)
63      axis.set_title("Numeric correlations in training data")
64      finish(figure, "06_correlations")
65
66      figure, axis = plt.subplots(figsize=(10, 4))
67      monthly = train.set_index("date")[TARGET].resample("MS").mean()
68      axis.plot(monthly.index, monthly.values, marker="o", color="#18857d")
69      axis.set(title="Mean revenue per observation by month (training sample)",
70               ylabel="Mean revenue (USD)", xlabel="Reporting month")
71      finish(figure, "07_monthly_sample")
72
```

Line 50: Save and close this figure using the local helper.

Line 52: Assign (figure, axes): create a Matplotlib figure and its axes.

Line 53: Repeat the indented steps for each (axis, column) from zip(axes, ['category', 'country', 'device']).

Line 54: Assign means: order table rows using the specified score or field.

Line 55: Draw a horizontal bar chart.

Line 56: Set the axis title, labels or other listed properties.

Line 57: Set the horizontal axis limits; a zero start prevents exaggerated bar differences.

Line 58: Save and close this figure using the local helper.

Line 60: Assign (figure, axis): create a Matplotlib figure and its axes.

Line 61-62: Draw the correlation matrix using annotated values and a fixed color scale.

Line 63: Label the chart with its subject and data population.

Line 64: Save and close this figure using the local helper.

Line 66: Assign (figure, axis): create a Matplotlib figure and its axes.

Line 67: Assign monthly: calculate an arithmetic average.

Line 68: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 69-70: Set the axis title, labels or other listed properties.

Line 71: Save and close this figure using the local helper.

charts.py | lines 73-96

```
    figure, axis = plt.subplots(figsize=(9, 4))
    ordered = comparison.sort_values("validation_rmse")
    axis.barh(ordered["model"], ordered["validation_rmse"], color="#18857d")
    axis.invert_yaxis()
    axis.set(xlabel="Validation RMSE (USD), lower is better", title="Five-model comparison")
    finish(figure, "08_model_comparison")

    sample = predictions.sample(min(3000, len(predictions)), random_state=42)
    figure, axes = plt.subplots(1, 2, figsize=(12, 4))
    axes[0].scatter(sample["actual"], sample["predicted"], s=8, alpha=0.3)
    low = min(sample["actual"].min(), sample["predicted"].min())
    high = max(sample["actual"].max(), sample["predicted"].max())
    axes[0].plot([low, high], [low, high], "--", color="#bd5c3c")
    axes[0].set(xlabel="Actual revenue (USD)", ylabel="Predicted revenue (USD)",
                title="Reserved test sample: ideal predictions lie on the line")
    axes[1].scatter(sample["predicted"], sample["actual"] - sample["predicted"],
                    s=8, alpha=0.3, color="#18857d")
    axes[1].axhline(0, linestyle="--", color="#bd5c3c")
    axes[1].set(xlabel="Predicted revenue (USD)", ylabel="Actual minus predicted (USD)",
                title="Reserved test residuals")
    finish(figure, "09_prediction_errors")

    figure, axis = plt.subplots(figsize=(9, 5))
    ordered = importance.sort_values("mae_increase_usd")
```

Line 73: Assign (figure, axis): create a Matplotlib figure and its axes.

Line 74: Assign ordered: order table rows using the specified score or field.

Line 75: Draw a horizontal bar chart.

Line 76: Put the best validation result at the top of the horizontal chart.

Line 77: Set the axis title, labels or other listed properties.

Line 78: Save and close this figure using the local helper.

Line 80: Assign sample: choose a reproducible subset for display or importance analysis.

Line 81: Assign (figure, axes): create a Matplotlib figure and its axes.

Line 82: Plot paired observations; transparency helps reveal dense regions.

Line 83: Assign low: find the smallest value.

Line 84: Assign high: find the largest value.

Line 85: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 86-87: Set the axis title, labels or other listed properties.

Line 88-89: Plot paired observations; transparency helps reveal dense regions.

Line 90: Draw the zero-residual reference line.

Line 91-92: Set the axis title, labels or other listed properties.

Line 93: Save and close this figure using the local helper.

Line 95: Assign (figure, axis): create a Matplotlib figure and its axes.

Line 96: Assign ordered: order table rows using the specified score or field.

charts.py | lines 97-101

```
    axis.barh(ordered["feature"], ordered["mae_increase_usd"], color="#18857d")
    axis.set(xlabel="Increase in validation MAE after shuffling (USD)",
             title="Feature importance before final refit; correlated inputs share effects")
    finish(figure, "10_feature_importance")
    return paths
```

Line 97: Draw a horizontal bar chart.

Line 98-99: Set the axis title, labels or other listed properties.

Line 100: Save and close this figure using the local helper.

Line 101: Return paths to the caller; this ends the current function call.

18. Code walkthrough: app.py

This file creates the Streamlit interface with ordinary Python functions. The model is loaded for inference; it is not trained during UI interactions.

app.py | lines 1-24

```
"""Run with: python -m streamlit run app.py"""

from importlib.metadata import version
import json

import joblib
import pandas as pd
import plotly.express as px
import streamlit as st

from utils import (ROOT, RAW_DATA, NUMERIC, CATEGORICAL, OPTIONAL, TARGET,
                   file_hash, load_data, predict_revenue, read_json)

st.set_page_config(page_title="Content Monetization Modeler", page_icon=" ",
                   layout="wide", initial_sidebar_state="auto")
px.defaults.template = "plotly_white"
px.defaults.color_discrete_sequence = ["#137F76", "#D28A53", "#6486A0", "#A69A62"]
LABELS = {"views": "Views", "likes": "Likes", "comments": "Comments",
          "watch_time_minutes": "Total watch time (minutes)",
          "video_length_minutes": "Video length (minutes)", "subscribers": "Subscribers",
          TARGET: "Ad revenue (USD)", "category": "Category", "country": "Country",
          "device": "Device"}


```

Line 3: Import version from importlib.metadata, providing installed package version information.

Line 4: Import json for readable metadata and report files.

Line 6: Import joblib for saving and loading fitted Python models.

Line 7: Import pandas as pd for DataFrames and CSV analysis.

Line 8: Import plotly.express as px for interactive figures.

Line 9: Import streamlit as st for the Python web interface and its tests.

Line 11-12: Import ROOT, RAW_DATA, NUMERIC, CATEGORICAL, OPTIONAL, TARGET, file_hash, load_data, predict_revenue, read_json from utils, providing shared project rules.

Line 14-15: Configure the browser title, layout and responsive sidebar behavior.

Line 16: Assign px.defaults.template: store this literal value as a setting or initial value.

Line 17: Assign px.defaults.color_discrete_sequence: collect these values in the specified order.

Line 18-22: Assign LABELS: build a dictionary mapping these names to their corresponding values.

app.py | lines 25-48

get_data: Streamlit cache_data stores the returned DataFrame and quality dictionary. The modification-time argument changes the cache key when the CSV changes, even though the loader reads the fixed project-relative source path.

get_model: Read metadata, check the model fingerprint and check the exact model-library versions. Only after these checks load the trusted joblib file. cache_resource reuses the fitted pipeline across ordinary widget interactions; modification-time arguments refresh it after regeneration.

model_assets: This is the application's model-loading boundary. File and metadata timestamps select a cache entry. A load failure becomes a visible recovery message followed by st.stop, which ends that app run without showing a raw traceback to the user.

```
@st.cache_data
def get_data(modified_time):
    """The timestamp makes Streamlit refresh its cache when the CSV changes."""
    return load_data()


@st.cache_resource
def get_model(model_time, metadata_time):
    """Load only the project's own model, checking its hash and package versions."""
```

```
    metadata = read_json(ROOT / "models" / "metadata.json")
    model_path = ROOT / "models" / "revenue_pipeline.joblib"
    if file_hash(model_path) != metadata["model_sha256"]:
        raise ValueError("Model and metadata do not match. Run python train_model.py.")
    for package, expected in metadata["versions"].items():
        if version(package) != expected:
            raise ValueError(f"Install requirements.txt: {package} must be {expected}.")
    return joblib.load(model_path), metadata


def model_assets():
    """Turn missing or incompatible model files into a helpful recovery message."""
    try:
        model_path = ROOT / "models" / "revenue_pipeline.joblib"
        metadata_path = ROOT / "models" / "metadata.json"
```

Line 25: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 26: Define get_data(modified_time). The indented body runs when this function is called.

Line 28: Return load_data() to the caller; this ends the current function call.

Line 31: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 32: Define get_model(model_time, metadata_time). The indented body runs when this function is called.

Line 34: Assign metadata: read saved JSON metadata or metrics as ordinary Python objects.

Line 35: Assign model_path: calculate the shown arithmetic or combine the indicated collections.

Line 36: Run the following indented branch only when file_hash(model_path) != metadata['model_sha256'] is true.

Line 37: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 38: Repeat the indented steps for each (package, expected) from metadata['versions'].items().

Line 39: Run the following indented branch only when version(package) != expected is true.

Line 40: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 41: Return (joblib.load(model_path), metadata) to the caller; this ends the current function call.

Line 44: Define model_assets(). The indented body runs when this function is called.

Line 46: Try the following operation; a matching exception is handled by the following recovery branch.

Line 47: Assign model_path: calculate the shown arithmetic or combine the indicated collections.

Line 48: Assign metadata_path: calculate the shown arithmetic or combine the indicated collections.

app.py | lines 49-72

plot: Apply a common Plotly layout, then display the figure at its container's width. This helper standardizes margins and fonts without a separate frontend framework.

overview: Show the project purpose, counts and sample mean, then a sampled scatter plot and navigation guidance. The expander keeps important source limitations available without crowding the main screen. All counts concern the deduplicated supplied source.

```
        return get_model(model_path.stat().st_mtime_ns, metadata_path.stat().st_mtime_ns)
    except Exception as error:
        st.error(f"The saved model is unavailable: {error}")
        st.info("From the project folder, install requirements.txt and run "
                "python train_model.py. Then refresh this page.")
        st.stop()


def plot(figure):
    figure.update_layout(margin=dict(l=15, r=15, t=50, b=20), height=370,
                         paper_bgcolor="rgba(0,0,0,0)", font=dict(size=13))
    st.plotly_chart(figure, use_container_width=True)


def overview(data, quality):
    st.caption("SOCIAL MEDIA ANALYTICS / PYTHON + MACHINE LEARNING")
    st.title("YouTube revenue, explained")
    st.write("Explore video performance and estimate advertising revenue with a "
             "model trained on the supplied learning dataset.")
    columns = st.columns(4)
    columns[0].metric("Observations", f"{len(data):,}")
    columns[1].metric("Video identifiers", f"{data.video_id.nunique():,}")
    columns[2].metric("Mean revenue / observation", f"${data[TARGET].mean():,.2f}")
    columns[3].metric("Exact duplicates removed", f"{quality['duplicates_removed']:,}")
```

Line 49: Return get_model(model_path.stat().st_mtime_ns, metadata_path.stat().st_mtime_ns) to the caller; this ends the current function call.

Line 50: Catch the named exception and enter the recovery or expected-error branch.

Line 51: Show the caught validation or file error without continuing as a successful prediction.

Line 52-53: Show an informational message.

Line 54: End this Streamlit execution after the recovery message.

Line 57: Define plot(figure). The indented body runs when this function is called.

Line 58-59: Call figure.update_layout with the shown arguments.

Line 60: Render the interactive Plotly figure at the container width.

Line 63: Define overview(data, quality). The indented body runs when this function is called.

Line 64: Display smaller contextual text.

Line 65: Display the main page title.

Line 66-67: Display explanatory text or values in the current Streamlit container.

Line 68: Assign columns: create side-by-side layout containers.

Line 69: Display a labeled numeric summary.

Line 70: Display a labeled numeric summary.

Line 71: Display a labeled numeric summary.

Line 72: Display a labeled numeric summary.

app.py | lines 73-96

```
73      st.divider()
74      left, right = st.columns([2, 1])
75      with left:
76          sample = data.sample(min(2000, len(data)), random_state=42)
77          plot(px.scatter(sample, x="watch_time_minutes", y=TARGET, color="category",
78                          opacity=0.45, labels=LABELS,
79                          title="Watch time and revenue in the supplied sample"))
80          st.caption("2,000 sampled rows, after exact duplicate removal. "
81                     "This is a descriptive relationship, not a causal result.")
82      with right:
83          st.subheader("Start with the data")
84          st.write("**Explore Data** — filter observations and inspect distributions.")
85          st.write("**Predict Revenue** — enter video metrics to estimate USD revenue.")
86          st.write("**Model Results** — understand validation scores and test errors.")
87          st.info("Synthetic learning data. Estimates apply to one reporting observation, "
88                  "not a video's lifetime earnings.")
89      with st.expander("Understand the dataset before making predictions"):
90          st.write("Views in the source range from 9,521 to 10,468. The prediction form "
91                   "accepts the numeric ranges used to fit the model. It does not "
92                   "extrapolate to very small or very large videos.")
93          st.write("Video identifiers repeat with inconsistent categories and lengths. "
94                   "We preserve distinct observations and keep each ID in one data split.")
95          st.write("Likes, comments and watch time each have 5% missing values after "
96                   "deduplication. The model uses fitted medians when these are unknown.")
```

Line 73: Separate the surrounding interface sections.

Line 74: Assign (left, right): create side-by-side layout containers.

Line 75: Enter left. The indented statements use this layout or test context.

Line 76: Assign sample: choose a reproducible subset for display or importance analysis.

Line 77-79: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 80-81: Display smaller contextual text.

Line 82: Enter right. The indented statements use this layout or test context.

Line 83: Display a section heading.

Line 84: Display explanatory text or values in the current Streamlit container.

Line 85: Display explanatory text or values in the current Streamlit container.

Line 86: Display explanatory text or values in the current Streamlit container.

Line 87-88: Show an informational message.

Line 89: Enter st.expander('Understand the dataset before making predictions'). The indented statements use this layout or test context.

Line 90-92: Display explanatory text or values in the current Streamlit container.

Line 93-94: Display explanatory text or values in the current Streamlit container.

Line 95-96: Display explanatory text or values in the current Streamlit container.

app.py | lines 97-120

explore: Read optional category filters and a reporting date range, then create a filtered DataFrame. Empty selections mean no restriction. Incomplete dates or no matches receive explanatory messages. Three tabs organize revenue patterns, distributions/quality and a data preview. CSV download contains every matching row even though only 200 are displayed.

```
 97
 98
 99  def explore(data):
100      st.title("Explore the dataset")
101      st.write("Descriptive charts use all deduplicated observations. Missing values "
102               "remain missing here; they are not replaced by model estimates.")
103      controls = st.columns(3)
104      selected = {}
105      for column, control in zip(CATEGORICAL, controls):
106          selected[column] = control.multiselect(LABELS[column],
107                                                 sorted(data[column].dropna().unique()))
108      dates = st.date_input("Reporting dates", value=(data.date.min().date(), data.date.max().date()),
109                            min_value=data.date.min().date(), max_value=data.date.max().date())
110      if len(dates) != 2:
111          st.info("Choose both a start date and an end date.")
112          return
113      filtered = data[(data.date.dt.date >= dates[0]) & (data.date.dt.date <= dates[1])]
114      for column in CATEGORICAL:
115          if selected[column]:
116              filtered = filtered[filtered[column].isin(selected[column])]
117      if filtered.empty:
118          st.info("No observations match these filters. Change a date or category filter.")
119          return
120      metrics = st.columns(3)
```

Line 99: Define explore(data). The indented body runs when this function is called.

Line 100: Display the main page title.

Line 101-102: Display explanatory text or values in the current Streamlit container.

Line 103: Assign controls: create side-by-side layout containers.

Line 104: Assign selected: build a dictionary mapping these names to their corresponding values.

Line 105: Repeat the indented steps for each (column, control) from zip(CATEGORICAL, controls).

Line 106-107: Assign selected[column]: show a multiple-choice control and obtain its selected values.

Line 108-109: Assign dates: show reporting-date controls and obtain the chosen date range.

Line 110: Run the following indented branch only when len(dates) != 2 is true.

Line 111: Show an informational message.

Line 112: Return control to the caller; this ends the current function call.

Line 113: Assign filtered: select the named field, columns or rows from the existing object.

Line 114: Repeat the indented steps for each column from CATEGORICAL.

Line 115: Run the following indented branch only when selected[column] is true.

Line 116: Assign filtered: select the named field, columns or rows from the existing object.

Line 117: Run the following indented branch only when filtered.empty is true.

Line 118: Show an informational message.

Line 119: Return control to the caller; this ends the current function call.

Line 120: Assign metrics: create side-by-side layout containers.

app.py | lines 121-144

```
121     metrics[0].metric("Matching observations", f"{len(filtered):,}")
122     metrics[1].metric("Mean revenue (USD)", f"${filtered[TARGET].mean():,.2f}")
123     metrics[2].metric("Median revenue (USD)", f"${filtered[TARGET].median():,.2f}")
124     first, second, third = st.tabs(["Revenue patterns", "Distributions & quality", "Data preview"])
125     with first:
126         left, right = st.columns(2)
127         with left:
128             group = st.selectbox("Compare by", CATEGORICAL, format_func=lambda x: LABELS[x])
129             means = filtered.groupby(group, as_index=False)[TARGET].mean()
130             plot(px.bar(means, x=group, y=TARGET, labels=LABELS, title="Mean revenue by group"))
131         with right:
132             sample = filtered.sample(min(2500, len(filtered)), random_state=42)
133             feature = st.selectbox("Compare revenue with", NUMERIC, format_func=lambda x: LABELS[x])
134             plot(px.scatter(sample, x=feature, y=TARGET, opacity=0.3, labels=LABELS,
135                             title="Revenue relationship (up to 2,500 rows)"))
136         monthly = filtered.set_index("date")[TARGET].resample("MS").mean().reset_index()
137         plot(px.line(monthly, x="date", y=TARGET, markers=True, labels=LABELS,
138                      title="Mean revenue per observation by month"))
139         st.caption("The date chart describes this sample; it does not establish "
140                    "growth or seasonality for a real channel.")
141     with second:
142         field = st.selectbox("Inspect a numeric field", [TARGET] + NUMERIC,
143                              format_func=lambda x: LABELS[x])
144         left, right = st.columns(2)
```

Line 121: Display a labeled numeric summary.

Line 122: Display a labeled numeric summary.

Line 123: Display a labeled numeric summary.

Line 124: Assign (first, second, third): create the named page tabs.

Line 125: Enter first. The indented statements use this layout or test context.

Line 126: Assign (left, right): create side-by-side layout containers.

Line 127: Enter left. The indented statements use this layout or test context.

Line 128: Assign group: show a one-choice dropdown and obtain the selection.

Line 129: Assign means: calculate an arithmetic average.

Line 130: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 131: Enter right. The indented statements use this layout or test context.

Line 132: Assign sample: choose a reproducible subset for display or importance analysis.

Line 133: Assign feature: show a one-choice dropdown and obtain the selection.

Line 134-135: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 136: Assign monthly: restore a simple row index.

Line 137-138: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 139-140: Display smaller contextual text.

Line 141: Enter second. The indented statements use this layout or test context.

Line 142-143: Assign field: show a one-choice dropdown and obtain the selection.

Line 144: Assign (left, right): create side-by-side layout containers.

app.py | lines 145-168

predict: Load the saved model, let the user mark three optional inputs unknown and render the remaining controls in a form. A submit validates the values and obtains one estimate. Errors return a clear message. Successful output includes input notes, the relevant held-out watch-time group's historical MAE and a JSON download. It does not claim a confidence interval.

```
145         with left:
146             plot(px.histogram(filtered, x=field, nbins=35, labels=LABELS, title="Distribution"))
147         with right:
148             plot(px.box(filtered, y=field, points=False, labels=LABELS, title="Spread and quartiles"))
149         missing = filtered.isna().sum().rename("Missing rows").to_frame()
150         st.dataframe(missing, width="stretch")
151         correlation = filtered[NUMERIC + [TARGET]].corr()
152         plot(px.imshow(correlation, text_auto=".2f", zmin=-1, zmax=1,
153                        color_continuous_scale="BrBG", title="Numeric correlations"))
```

```
    with third:
        st.caption("Preview shows the first 200 matching rows. Download includes all matching rows.")
        st.dataframe(filtered.head(200), hide_index=True, width="stretch")
        st.download_button("Download filtered CSV", filtered.to_csv(index=False).encode("utf-8"),
                           "filtered_youtube_data.csv", "text/csv")


def predict():
    st.title("Estimate advertising revenue")
    st.write("Enter metrics for one reporting observation. Use the same reporting "
             "period for views, interactions and total watch time.")
    pipeline, metadata = model_assets()
    st.caption(f"Model: {metadata['model']} · Revenue currency: USD")
    unknown = st.multiselect("Mark inputs you do not know", OPTIONAL,
                             format_func=lambda x: LABELS[x])
```

Line 145: Enter left. The indented statements use this layout or test context.

Line 146: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 147: Enter right. The indented statements use this layout or test context.

Line 148: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 149: Assign missing: call filtered.isna().sum().rename('Missing rows').to_frame with the shown arguments.

Line 150: Render a table with the specified visible columns and layout.

Line 151: Assign correlation: calculate numeric pairwise correlations.

Line 152-153: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 154: Enter third. The indented statements use this layout or test context.

Line 155: Display smaller contextual text.

Line 156: Render a table with the specified visible columns and layout.

Line 157-158: Offer the provided CSV or JSON content as a downloadable file.

Line 161: Define predict(). The indented body runs when this function is called.

Line 162: Display the main page title.

Line 163-164: Display explanatory text or values in the current Streamlit container.

Line 165: Assign (pipeline, metadata): load the checked model and metadata or display a recovery message.

Line 166: Display smaller contextual text.

Line 167-168: Assign unknown: show a multiple-choice control and obtain its selected values.

app.py | lines 169-192

```
    values = {}
    with st.form("revenue_form"):
        left, right = st.columns(2)
        for index, column in enumerate(NUMERIC):
            panel = left if index < 3 else right
            low, high = metadata["ranges"][column]
            integer = column in ["views", "likes", "comments", "subscribers"]
            default = metadata["defaults"][column]
            value = panel.number_input(LABELS[column], min_value=0 if integer else 0.0,
                                       value=int(default) if integer else round(default, 2),
                                       step=1 if integer else 0.1, disabled=column in unknown,
                                       help=f"Fitted range: {low:,.2f} to {high:,.2f}",
                                       key="input_" + column)
            values[column] = None if column in unknown else value
        columns = st.columns(3)
        for column, panel in zip(CATEGORICAL, columns):
            values[column] = panel.selectbox(LABELS[column], metadata["categories"][column])
        submitted = st.form_submit_button("Predict revenue", type="primary", width="stretch")
    if submitted:
        try:
            revenue, notes = predict_revenue(pipeline, values, metadata)
        except ValueError as error:
            st.error(str(error))
            return
```

Line 169: Assign values: build a dictionary mapping these names to their corresponding values.

Line 170: Enter st.form('revenue_form'). The indented statements use this layout or test context.

Line 171: Assign (left, right): create side-by-side layout containers.

Line 172: Repeat the indented steps for each (index, column) from enumerate(NUMERIC).

Line 173: Assign panel: choose the first value when the condition is true, otherwise the alternative.

Line 174: Assign (low, high): select the named field, columns or rows from the existing object.

Line 175: Assign integer: evaluate the comparison, producing a Boolean value or mask.

Line 176: Assign default: select the named field, columns or rows from the existing object.

Line 177-181: Assign value: show a numeric control and obtain the entered value.

Line 182: Assign values[column]: choose the first value when the condition is true, otherwise the alternative.

Line 183: Assign columns: create side-by-side layout containers.

Line 184: Repeat the indented steps for each (column, panel) from zip(CATEGORICAL, columns).

Line 185: Assign values[column]: show a one-choice dropdown and obtain the selection.

Line 186: Assign submitted: submit the grouped form values and report whether it was pressed.

Line 187: Run the following indented branch only when submitted is true.

Line 188: Try the following operation; a matching exception is handled by the following recovery branch.

Line 189: Assign (revenue, notes): validate a single input row and return its predicted revenue plus notes.

Line 190: Catch the named exception and enter the recovery or expected-error branch.

Line 191: Show the caught validation or file error without continuing as a successful prediction.

Line 192: Return control to the caller; this ends the current function call.

app.py | lines 193-216

model_results: Display the selected model's reserved test scores separately from the five-model validation comparison. Show missing-watch-time subgroup errors and saved diagnostic images. Explain importance limitations and display the ID/row counts of each partition.

```
193          with st.container(border=True):
194              st.metric("Estimated ad revenue (USD)", f"${revenue:,.2f}")
195              st.caption("Estimate for this reporting observation. It is not a guaranteed payment.")
196          for note in notes:
197              st.warning(note)
198          scores = read_json(ROOT / "reports" / "test_metrics.json")
199          group = "missing" if "watch_time_minutes" in unknown else "observed"
200          error = scores["watch_time_groups"][group]["mae"]
201          st.info(f"In held-out test rows with {group} watch time, mean absolute error "
202                  f"was ${error:.2f}. This is historical average error, not a confidence interval.")
203          payload = {"inputs": values, "estimated_ad_revenue_usd": revenue, "model": metadata["model"]}
204          st.download_button("Download this estimate", json.dumps(payload, indent=2),
205                             "revenue_estimate.json", "application/json")
206      with st.expander("Supported ranges and missing inputs"):
207          ranges = pd.DataFrame(metadata["ranges"], index=["Minimum", "Maximum"]).T
208          st.dataframe(ranges, width="stretch")
209          st.write("Numeric inputs outside these ranges are rejected. Inputs within the "
210                   "individual ranges can still form an unusual combination. The synthetic "
211                   "training data does not validate estimates for real channels.")
212          st.write("Unknown values use the medians saved during training. In particular, "
213                   "missing watch time makes estimates much less informative.")
214
215
216  def model_results():
```

Line 193: Enter st.container(border=True). The indented statements use this layout or test context.

Line 194: Display a labeled numeric summary.

Line 195: Display smaller contextual text.

Line 196: Repeat the indented steps for each note from notes.

Line 197: Show a relevant limitation or recovery warning.

Line 198: Assign scores: read saved JSON metadata or metrics as ordinary Python objects.

Line 199: Assign group: choose the first value when the condition is true, otherwise the alternative.

Line 200: Assign error: select the named field, columns or rows from the existing object.

Line 201-202: Show an informational message.

Line 203: Assign payload: build a dictionary mapping these names to their corresponding values.

Line 204-205: Offer the provided CSV or JSON content as a downloadable file.

Line 206: Enter st.expander('Supported ranges and missing inputs'). The indented statements use this layout or test context.

Line 207: Assign ranges: read the indicated property from the existing object.

Line 208: Render a table with the specified visible columns and layout.

Line 209-211: Display explanatory text or values in the current Streamlit container.

Line 212-213: Display explanatory text or values in the current Streamlit container.

Line 216: Define model_results(). The indented body runs when this function is called.

app.py | lines 217-240

```
217     st.title("Model comparison & results")
218     pipeline, metadata = model_assets()
219     scores = read_json(ROOT / "reports" / "test_metrics.json")
220     comparison = pd.read_csv(ROOT / "reports" / "model_comparison.csv")
221     st.write(f"**Selected model: {metadata['model']}.** {metadata['selection_rule']}.")
222     st.caption("These headline metrics use the reserved test set after the selected "
223                "model was refitted on training + validation data.")
224     columns = st.columns(3)
225     columns[0].metric("Test MAE (USD)", f"${scores['mae']:.2f}")
226     columns[1].metric("Test RMSE (USD)", f"${scores['rmse']:.2f}")
227     columns[2].metric("Test R²", f"{scores['r2']:.4f}")
228     with st.expander("What do these scores mean?"):
229         st.write("MAE is the average size of a prediction error in dollars. RMSE gives "
230                  "larger errors extra weight. Lower is better for both. R² compares "
231                  "the model with a mean prediction; it is not an accuracy percentage.")
232         st.write(f"A simple mean-only reference has test MAE ${scores['mean_baseline']['mae']:.2f}.")
233     st.subheader("Validation comparison")
234     plot(px.bar(comparison.sort_values("validation_rmse"), x="model", y="validation_rmse",
235                 labels={"model": "Model", "validation_rmse": "Validation RMSE (USD)"},
236                 title="All five models used the same training and validation partitions"))
237     st.dataframe(comparison.drop(columns="fit_and_score_seconds").round(4),
238                  hide_index=True, width="stretch")
239     st.subheader("Why missing watch time matters")
240     groups = pd.DataFrame(scores["watch_time_groups"]).T
```

Line 217: Display the main page title.

Line 218: Assign (pipeline, metadata): load the checked model and metadata or display a recovery message.

Line 219: Assign scores: read saved JSON metadata or metrics as ordinary Python objects.

Line 220: Assign comparison: read a CSV into a Pandas DataFrame.

Line 221: Display explanatory text or values in the current Streamlit container.

Line 222-223: Display smaller contextual text.

Line 224: Assign columns: create side-by-side layout containers.

Line 225: Display a labeled numeric summary.

Line 226: Display a labeled numeric summary.

Line 227: Display a labeled numeric summary.

Line 228: Enter st.expander('What do these scores mean?'). The indented statements use this layout or test context.

Line 229-231: Display explanatory text or values in the current Streamlit container.

Line 232: Display explanatory text or values in the current Streamlit container.

Line 233: Display a section heading.

Line 234-236: Draw the supplied line or render a shared Plotly figure, according to this call.

Line 237-238: Render a table with the specified visible columns and layout.

Line 239: Display a section heading.

Line 240: Assign groups: read the indicated property from the existing object.

app.py | lines 241-264

report_image: Check that a generated PNG exists before asking Streamlit to load it. If absent, explain how to rebuild charts rather than displaying a broken media exception.

main: Render the sidebar once, load data through the cache and dispatch to the selected page function. Expected data/report failures are caught at the application boundary and explained with a recovery instruction. The __main__ guard makes direct execution call this function.

```
241      st.dataframe(groups.round(4), width="stretch")
242      st.caption("Rows are grouped by whether watch time was originally observed. "
243                 "Imputation cannot recover information that was never recorded.")
244      st.subheader("Prediction errors")
245      report_image("09_prediction_errors.png")
246      st.subheader("Which inputs help the model?")
247      report_image("10_feature_importance.png")
248      st.caption("Importance was measured on validation rows before refitting. It measures "
249                 "predictive reliance, not causation; correlated features can share information.")
250      st.subheader("Data separation")
251      st.dataframe(pd.DataFrame(metadata["split"]).T, width="stretch")
252      st.write("Each video ID belongs to exactly one partition. The target, date and video ID "
253               "are excluded from model inputs. Test rows did not choose the model or medians.")
254
255
256  def report_image(filename):
257      path = ROOT / "reports" / "figures" / filename
258      if path.exists():
259          st.image(str(path), width="stretch")
260      else:
261          st.warning(f"Missing chart: {filename}. Run python train_model.py to rebuild it.")
262
263
264  def main():
```

Line 241: Render a table with the specified visible columns and layout.

Line 242-243: Display smaller contextual text.

Line 244: Display a section heading.

Line 245: Call report_image with the shown arguments.

Line 246: Display a section heading.

Line 247: Call report_image with the shown arguments.

Line 248-249: Display smaller contextual text.

Line 250: Display a section heading.

Line 251: Render a table with the specified visible columns and layout.

Line 252-253: Display explanatory text or values in the current Streamlit container.

Line 256: Define report_image(filename). The indented body runs when this function is called.

Line 257: Assign path: calculate the shown arithmetic or combine the indicated collections.

Line 258: Run the following indented branch only when path.exists() is true.

Line 259: Render the saved chart or screenshot.

Line 260: Take this alternative branch when the preceding condition is false.

Line 261: Show a relevant limitation or recovery warning.

Line 264: Define main(). The indented body runs when this function is called.

app.py | lines 265-288

```
265      st.sidebar.title("Content Monetization")
266      st.sidebar.caption("YouTube revenue modeler")
267      page = st.sidebar.radio("Navigation", ["Overview", "Explore Data", "Predict Revenue", "Model
     Results"])
268      st.sidebar.divider()
269      st.sidebar.caption("Python · Pandas · Scikit-learn · Streamlit")
270      st.sidebar.info("Learning project using synthetic data. All revenue values are in USD.")
271      try:
272          data, quality = get_data(RAW_DATA.stat().st_mtime_ns)
273          if page == "Overview":
274              overview(data, quality)
275          elif page == "Explore Data":
276              explore(data)
277          elif page == "Predict Revenue":
278              predict()
279          else:
280              model_results()
281      except (OSError, ValueError, KeyError) as error:
282          st.error(f"A project file needs attention: {error}")
283          st.info("Keep the data, models and reports folders together. Restore the supplied "
284                  "CSV or run python train_model.py to regenerate the reports.")
285
286
287  if __name__ == "__main__":
288      main()
```

Line 265: Display the main page title.

Line 266: Display smaller contextual text.

Line 267: Assign page: show navigation choices and obtain the current page.

Line 268: Separate the surrounding interface sections.

Line 269: Display smaller contextual text.

Line 270: Show an informational message.

Line 271: Try the following operation; a matching exception is handled by the following recovery branch.

Line 272: Assign (data, quality): retrieve the cached validated dataset and its quality summary.

Line 273: Run the following indented branch only when page == 'Overview' is true.

Line 274: Call overview with the shown arguments.

Line 275: Run the following indented branch only when page == 'Explore Data' is true.

Line 276: Call explore with the shown arguments.

Line 277: Run the following indented branch only when page == 'Predict Revenue' is true.

Line 278: Apply the fitted pipeline to obtain revenue estimates.

Line 279: Take this alternative branch when the preceding condition is false.

Line 280: Call model_results with the shown arguments.

Line 281: Catch the named exception and enter the recovery or expected-error branch.

Line 282: Show the caught validation or file error without continuing as a successful prediction.

Line 283-284: Show an informational message.

Line 287: Run this entry point only when the file is executed directly, rather than imported.

Line 288: Call main with the shown arguments.

19. Notebook cell-by-cell guide

Notebook cell	Purpose
3	1. Imports and project folder
5	2. Load and validate the data
7	3. Missing values and synthetic-data issues
9	4. Separate training, validation and test data
11	5. Numerical exploration on the training partition
13	6. Relationships, groups and reporting dates
15	7. Two understandable engineered features
17	8. Preprocessing and five regression models
19	9. Read the errors correctly
21	10. Visual analysis and model diagnostics
23	11. Feature importance without using test data for selection
25	12. Load the saved pipeline and predict
27	13. Missing inputs and rejected extrapolation

Markdown cells explain the adjacent code and are included in the notebook itself. The code-cell listings below have local line numbers within each cell. Their outputs, including tables and figures, are already saved in the executed notebook.

Notebook cell 3: 1. Imports and project folder

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 3 | lines 1-18

```
 1  from pathlib import Path
 2  import sys
 3  import joblib
 4  import numpy as np
 5  import pandas as pd
 6  from IPython.display import display, Image
 7
 8  project_root = Path.cwd()
 9  if not (project_root / "utils.py").exists():
10      project_root = project_root.parent
11  if not (project_root / "utils.py").exists():
12      raise FileNotFoundError("Open this notebook inside the supplied project folder.")
13  sys.path.insert(0, str(project_root))
14
15  from utils import (INPUTS, NUMERIC, TARGET, load_data, split_data, add_features,
16                     predict_revenue, read_json)
17  from train_model import run_training
18  print("Project folder:", project_root.name)
```

Line 1: Import Path from pathlib, providing portable file paths.

Line 2: Import sys for the current interpreter and module search path.

Line 3: Import joblib for saving and loading fitted Python models.

Line 4: Import numpy as np for numeric arrays and finite-value checks.

Line 5: Import pandas as pd for DataFrames and CSV analysis.

Line 6: Import display, Image from IPython.display, providing the named reusable components.

Line 8: Assign project_root: call Path.cwd with the shown arguments.

Line 9: Run the following indented branch only when not (project_root / 'utils.py').exists() is true.

Line 10: Assign project_root: read the indicated property from the existing object.

Line 11: Run the following indented branch only when not (project_root / 'utils.py').exists() is true.

Line 12: Raise the stated error with a useful message. The caller can catch it, and tests can check that invalid input is rejected.

Line 13: Add the project root to the module search path so shared files can be imported.

Line 15-16: Import INPUTS, NUMERIC, TARGET, load_data, split_data, add_features, predict_revenue, read_json from utils, providing shared project rules.

Line 17: Import run_training from train_model, providing the training workflow.

Line 18: Write the status or result to the terminal or notebook output.

Notebook cell 5: 2. Load and validate the data

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 5 | lines 1-4

```
1  data, quality = load_data()
2  display(pd.Series(quality).to_frame("Result"))
3  display(data.head())
4  display(data.dtypes.to_frame("Data type"))
```

Line 1: Assign (data, quality): load, validate and deduplicate the source, returning data and quality counts.

Line 2: Call display with the shown arguments.

Line 3: Call display with the shown arguments.

Line 4: Call display with the shown arguments.

Notebook cell 7: 3. Missing values and synthetic-data issues

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 7 | lines 1-9

```
1  missing = data.isna().sum().to_frame("Missing rows")
2  missing["Percent"] = missing["Missing rows"] / len(data) * 100
3  display(missing)
4  print("Repeated video/date pairs beyond the first:",
5        data.duplicated(["video_id", "date"]).sum())
6  print("IDs with multiple categories:",
7        (data.groupby("video_id")["category"].nunique() > 1).sum())
8  print("Average watch time greater than video length:",
9        ((data["watch_time_minutes"] / data["views"]) > data["video_length_minutes"]).sum())
```

Line 1: Assign missing: call data.isna().sum().to_frame with the shown arguments.

Line 2: Assign missing['Percent']: calculate the shown arithmetic or combine the indicated collections.

Line 3: Call display with the shown arguments.

Line 4-5: Write the status or result to the terminal or notebook output.

Line 6-7: Write the status or result to the terminal or notebook output.

Line 8-9: Write the status or result to the terminal or notebook output.

Notebook cell 9: 4. Separate training, validation and test data

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 9 | lines 1-8

```
1  train, validation, test = split_data(data)
2  parts = {"train": train, "validation": validation, "test": test}
3  display(pd.DataFrame({name: {"rows": len(frame), "video_ids": frame.video_id.nunique()}
4                        for name, frame in parts.items()}).T)
5  assert set(train.video_id).isdisjoint(validation.video_id)
6  assert set(train.video_id).isdisjoint(test.video_id)
7  assert set(validation.video_id).isdisjoint(test.video_id)
8  print("No video IDs overlap between partitions.")
```

Line 1: Assign (train, validation, test): separate observations into training, validation and test by video ID.

Line 2: Assign parts: build a dictionary mapping these names to their corresponding values.

Line 3-4: Call display with the shown arguments.

Line 5: Verify set(train.video_id).isdisjoint(validation.video_id). A false condition fails this check rather than silently continuing.

Line 6: Verify set(train.video_id).isdisjoint(test.video_id). A false condition fails this check rather than silently continuing.

Line 7: Verify set(validation.video_id).isdisjoint(test.video_id). A false condition fails this check rather than silently continuing.

Line 8: Write the status or result to the terminal or notebook output.

Notebook cell 11: 5. Numerical exploration on the training partition

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 11 | lines 1-8

```
display(train[NUMERIC + [TARGET]].describe().round(2))
outliers = {}
for column in NUMERIC + [TARGET]:
    q1, q3 = train[column].quantile([0.25, 0.75])
    iqr = q3 - q1
    outliers[column] = ((train[column] < q1 - 1.5 * iqr) |
                        (train[column] > q3 + 1.5 * iqr)).sum()
display(pd.Series(outliers, name="IQR flags (not automatic deletions)"))
```

Line 1: Call display with the shown arguments.

Line 2: Assign outliers: build a dictionary mapping these names to their corresponding values.

Line 3: Repeat the indented steps for each column from NUMERIC + [TARGET].

Line 4: Assign (q1, q3): calculate the requested distribution quartiles.

Line 5: Assign iqr: calculate the shown arithmetic or combine the indicated collections.

Line 6-7: Assign outliers[column]: call ((train[column] < q1 - 1.5 * iqr) | (train[column] > q3 + 1.5 * iqr)).sum with the shown arguments.

Line 8: Call display with the shown arguments.

Notebook cell 13: 6. Relationships, groups and reporting dates

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 13 | lines 1-5

```
1  display(train[NUMERIC + [TARGET]].corr().round(4))
2  for column in ["category", "country", "device"]:
3      display(train.groupby(column)[TARGET].agg(["count", "mean", "median"]).round(2))
4  monthly = train.set_index("date")[TARGET].resample("MS").agg(["count", "mean"])
5  display(monthly.round(2))
```

Line 1: Call display with the shown arguments.

Line 2: Repeat the indented steps for each column from ['category', 'country', 'device'].

Line 3: Call display with the shown arguments.

Line 4: Assign monthly: call train.set_index('date')[TARGET].resample('MS').agg with the shown arguments.

Line 5: Call display with the shown arguments.

Notebook cell 15: 7. Two understandable engineered features

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 15 | lines 1-7

```
1  display(add_features(train).head())
2  example = train[INPUTS].head(1).copy()
3  example["views"] = 10000
4  example["likes"] = 1000
5  example["comments"] = 200
6  example["watch_time_minutes"] = 30000
7  display(add_features(example)[["engagement_rate", "average_watch_minutes"]])
```

Line 1: Call display with the shown arguments.

Line 2: Assign example: make a separate copy so later edits do not alter the source object.

Line 3: Assign example['views']: store this literal value as a setting or initial value.

Line 4: Assign example['likes']: store this literal value as a setting or initial value.

Line 5: Assign example['comments']: store this literal value as a setting or initial value.

Line 6: Assign example['watch_time_minutes']: store this literal value as a setting or initial value.

Line 7: Call display with the shown arguments.

Notebook cell 17: 8. Preprocessing and five regression models

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 17 | lines 1-3

```
1  comparison, test_scores = run_training()
2  display(comparison.round(4))
3  display(pd.Series({key: test_scores[key] for key in ["model", "mae", "rmse", "r2", "test_rows"]}))
```

Line 1: Assign (comparison, test_scores): call run_training with the shown arguments.

Line 2: Call display with the shown arguments.

Line 3: Call display with the shown arguments.

Notebook cell 19: 9. Read the errors correctly

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 19 | lines 1-3

```
1  display(pd.DataFrame(test_scores["watch_time_groups"]).T.round(4))
2  display(pd.Series(test_scores["mean_baseline"], name="Mean-only test reference"))
3  print("Large errors concentrate among observations with missing watch time.")
```

Line 1: Call display with the shown arguments.

Line 2: Call display with the shown arguments.

Line 3: Write the status or result to the terminal or notebook output.

Notebook cell 21: 10. Visual analysis and model diagnostics

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 21 | lines 1-3

```
1  for path in sorted((project_root / "reports" / "figures").glob("*.png")):
2      print(path.stem)
3      display(Image(filename=str(path), width=850))
```

Line 1: Repeat the indented steps for each path from sorted((project_root / 'reports' / 'figures').glob('*.png')).

Line 2: Write the status or result to the terminal or notebook output.

Line 3: Call display with the shown arguments.

Notebook cell 23: 11. Feature importance without using test data for selection

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 23 | lines 1-2

```
1  importance = pd.read_csv(project_root / "reports" / "feature_importance.csv")
2  display(importance.sort_values("mae_increase_usd", ascending=False).round(4))
```

Line 1: Assign importance: read a CSV into a Pandas DataFrame.

Line 2: Call display with the shown arguments.

Notebook cell 25: 12. Load the saved pipeline and predict

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 25 | lines 1-8

```
1  pipeline = joblib.load(project_root / "models" / "revenue_pipeline.joblib")
2  metadata = read_json(project_root / "models" / "metadata.json")
3  inputs = metadata["defaults"].copy()
4  for column, choices in metadata["categories"].items():
5      inputs[column] = choices[0]
6  revenue, notes = predict_revenue(pipeline, inputs, metadata)
7  print(f"Example estimate: ${revenue:.2f}")
8  print("Input notes:", notes)
```

Line 1: Assign pipeline: restore the fitted pipeline from the trusted saved artifact.

Line 2: Assign metadata: read saved JSON metadata or metrics as ordinary Python objects.

Line 3: Assign inputs: make a separate copy so later edits do not alter the source object.

Line 4: Repeat the indented steps for each (column, choices) from metadata['categories'].items().

Line 5: Assign inputs[column]: select the named field, columns or rows from the existing object.

Line 6: Assign (revenue, notes): validate a single input row and return its predicted revenue plus notes.

Line 7: Write the status or result to the terminal or notebook output.

Line 8: Write the status or result to the terminal or notebook output.

Notebook cell 27: 13. Missing inputs and rejected extrapolation

The following code is reproduced exactly from the delivered notebook or supporting file.

Notebook cell 27 | lines 1-12

```
unknown_watch = inputs.copy()
unknown_watch["watch_time_minutes"] = None
value, notes = predict_revenue(pipeline, unknown_watch, metadata)
print(f"Estimate with unknown watch time: ${value:.2f}")
print(notes)

unsupported = inputs.copy()
unsupported["views"] = 1000000
try:
    predict_revenue(pipeline, unsupported, metadata)
except ValueError as error:
    print("Expected validation message:", error)
```

Line 1: Assign unknown_watch: make a separate copy so later edits do not alter the source object.

Line 2: Assign unknown_watch['watch_time_minutes']: store this literal value as a setting or initial value.

Line 3: Assign (value, notes): validate a single input row and return its predicted revenue plus notes.

Line 4: Write the status or result to the terminal or notebook output.

Line 5: Write the status or result to the terminal or notebook output.

Line 7: Assign unsupported: make a separate copy so later edits do not alter the source object.

Line 8: Assign unsupported['views']: store this literal value as a setting or initial value.

Line 9: Try the following operation; a matching exception is handled by the following recovery branch.

Line 10: Validate a single input row and return its predicted revenue plus notes.

Line 11: Catch the named exception and enter the recovery or expected-error branch.

Line 12: Write the status or result to the terminal or notebook output.

20. Automated tests: complete code

These tests validate outcomes and important failure cases. Fixtures share expensive read-only resources; parameterization runs one rule against several inputs.

tests/test_project.py | lines 1-24

```
"""Data, leakage, saved-model, invalid-input and Streamlit regression checks."""

import sys
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import pytest
from sklearn.linear_model import LinearRegression
from streamlit.testing.v1 import AppTest

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import utils
from train_model import regression_metrics
from utils import (ROOT, INPUTS, TARGET, NUMERIC, DERIVED, add_features, build_pipeline,
                   file_hash, load_data, predict_revenue, read_json, split_data)


@pytest.fixture(scope="module")
def data():
    return load_data()[0]


```

Line 3: Import sys for the current interpreter and module search path.

Line 4: Import Path from pathlib, providing portable file paths.

Line 6: Import joblib for saving and loading fitted Python models.

Line 7: Import numpy as np for numeric arrays and finite-value checks.

Line 8: Import pandas as pd for DataFrames and CSV analysis.

Line 9: Import pytest for automated checks.

Line 10: Import LinearRegression from sklearn.linear_model, providing machine-learning models and preparation.

Line 11: Import AppTest from streamlit.testing.v1, providing the Python web interface and its tests.

Line 13: Add the project root to the module search path so shared files can be imported.

Line 14: Import utils for shared project rules.

Line 15: Import regression_metrics from train_model, providing the training workflow.

Line 16-17: Import ROOT, INPUTS, TARGET, NUMERIC, DERIVED, add_features, build_pipeline, file_hash, load_data, predict_revenue, read_json, split_data from utils, providing shared project rules.

Line 20: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 21: Define data(). The indented body runs when this function is called.

Line 22: Return load_data()[0] to the caller; this ends the current function call.

tests/test_project.py | lines 25-48

test_source_quality_and_exact_duplicates: This test checks source quality and exact duplicates. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

```
@pytest.fixture(scope="module")
def model():
    return joblib.load(ROOT / "models" / "revenue_pipeline.joblib")


@pytest.fixture(scope="module")
def metadata():
    return read_json(ROOT / "models" / "metadata.json")

```

```
34
35  @pytest.fixture
36  def valid_input(metadata):
37      values = metadata["defaults"].copy()
38      for column, choices in metadata["categories"].items():
39          values[column] = choices[0]
40      return values
41
42
43  def test_source_quality_and_exact_duplicates(data):
44      frame, quality = load_data()
45      assert quality["raw_rows"] == 122400
46      assert quality["duplicates_removed"] == 2400
47      assert len(frame) == 120000 and not frame.duplicated().any()
48      assert data["likes"].isna().sum() == 6000
```

Line 25: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 26: Define model(). The indented body runs when this function is called.

Line 27: Return joblib.load(ROOT / 'models' / 'revenue_pipeline.joblib') to the caller; this ends the current function call.

Line 30: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 31: Define metadata(). The indented body runs when this function is called.

Line 32: Return read_json(ROOT / 'models' / 'metadata.json') to the caller; this ends the current function call.

Line 35: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 36: Define valid_input(metadata). The indented body runs when this function is called.

Line 37: Assign values: make a separate copy so later edits do not alter the source object.

Line 38: Repeat the indented steps for each (column, choices) from metadata['categories'].items().

Line 39: Assign values[column]: select the named field, columns or rows from the existing object.

Line 40: Return values to the caller; this ends the current function call.

Line 43: Define test_source_quality_and_exact_duplicates(data). The indented body runs when this function is called.

Line 44: Assign (frame, quality): load, validate and deduplicate the source, returning data and quality counts.

Line 45: Verify quality['raw_rows'] == 122400. A false condition fails this check rather than silently continuing.

Line 46: Verify quality['duplicates_removed'] == 2400. A false condition fails this check rather than silently continuing.

Line 47: Verify len(frame) == 120000 and (not frame.duplicated().any()). A false condition fails this check rather than silently continuing.

Line 48: Verify data['likes'].isna().sum() == 6000. A false condition fails this check rather than silently continuing.

tests/test_project.py | lines 49-72

test_group_split_has_no_overlap_and_is_reproducible: This test checks group split has no overlap and is reproducible. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_invalid_source_rejected: This test checks invalid source rejected. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

```
49      assert data.duplicated(["video_id", "date"]).sum() == 3849
50
51
52  def test_group_split_has_no_overlap_and_is_reproducible(data):
53      train, validation, test = split_data(data)
54      ids = [set(part.video_id) for part in [train, validation, test]]
55      assert not ids[0] & ids[1] and not ids[0] & ids[2] and not ids[1] & ids[2]
56      assert len(train) + len(validation) + len(test) == len(data)
57      for first, second in zip([train, validation, test], split_data(data)):
58          pd.testing.assert_frame_equal(first, second)
59
60
61  @pytest.mark.parametrize("column,value", [("views", -1), ("likes", np.inf),
62      ("comments", 1.5), ("views", 0), ("video_length_minutes", 0),
63      (TARGET, np.nan), ("date", "not-a-date"), ("video_id", " ")])
64  def test_invalid_source_rejected(data, tmp_path, column, value):
65      frame = data.head(20).copy()
66      if column == "date":
67          frame[column] = frame[column].astype(object)
68      frame.loc[0, column] = value
69      path = tmp_path / "bad.csv"
70      frame.to_csv(path, index=False)
71      with pytest.raises((ValueError, TypeError)):
```

```
72          load_data(path)
```

Line 49: Verify data.duplicated(['video_id', 'date']).sum() == 3849. A false condition fails this check rather than silently continuing.

Line 52: Define test_group_split_has_no_overlap_and_is_reproducible(data). The indented body runs when this function is called.

Line 53: Assign (train, validation, test): separate observations into training, validation and test by video ID.

Line 54: Assign ids: build a list by applying the expression to each matching item.

Line 55: Verify not ids[0] & ids[1] and (not ids[0] & ids[2]) and (not ids[1] & ids[2]). A false condition fails this check rather than silently continuing.

Line 56: Verify len(train) + len(validation) + len(test) == len(data). A false condition fails this check rather than silently continuing.

Line 57: Repeat the indented steps for each (first, second) from zip([train, validation, test], split_data(data)).

Line 58: Verify that both DataFrames agree in values and structure.

Line 61: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 64: Define test_invalid_source_rejected(data, tmp_path, column, value). The indented body runs when this function is called.

Line 65: Assign frame: make a separate copy so later edits do not alter the source object.

Line 66: Run the following indented branch only when column == 'date' is true.

Line 67: Assign frame[column]: convert the values to the requested data type.

Line 68: Assign frame.loc[0, column]: reuse the existing value named value.

Line 69: Assign path: calculate the shown arithmetic or combine the indicated collections.

Line 70: Write the table to CSV without adding a redundant row-index column.

Line 71: Enter pytest.raises((ValueError, TypeError)). The indented statements use this layout or test context.

Line 72: Load, validate and deduplicate the source, returning data and quality counts.

tests/test_project.py | lines 73-96

test_missing_schema_and_empty_file: This test checks missing schema and empty file. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_feature_math_no_target_leak_and_zero_division: This test checks feature math no target leak and zero division. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

```
73
74
75  def test_missing_schema_and_empty_file(data, tmp_path):
76      path = tmp_path / "bad.csv"
77      data.head(2).drop(columns=[TARGET]).to_csv(path, index=False)
78      with pytest.raises(ValueError, match="Missing columns"):
79          load_data(path)
80      data.head(0).to_csv(path, index=False)
81      with pytest.raises(ValueError, match="no rows"):
82          load_data(path)
83
84
85  def test_feature_math_no_target_leak_and_zero_division(data):
86      row = data.head(1).copy()
87      row.loc[0, ["views", "likes", "comments", "watch_time_minutes"]] = [100, 10, 5, 250]
88      features = add_features(row)
89      assert features.loc[0, "engagement_rate"] == 0.15
90      assert features.loc[0, "average_watch_minutes"] == 2.5
91      assert set(features.columns) == set(INPUTS + DERIVED)
92      row.loc[0, TARGET] = 999999
93      pd.testing.assert_frame_equal(features, add_features(row))
94      row.loc[0, "views"] = 0
95      assert add_features(row)[DERIVED].isna().all().all()
96
```

Line 75: Define test_missing_schema_and_empty_file(data, tmp_path). The indented body runs when this function is called.

Line 76: Assign path: calculate the shown arithmetic or combine the indicated collections.

Line 77: Write the table to CSV without adding a redundant row-index column.

Line 78: Enter pytest.raises(ValueError, match='Missing columns'). The indented statements use this layout or test context.

Line 79: Load, validate and deduplicate the source, returning data and quality counts.

Line 80: Write the table to CSV without adding a redundant row-index column.

Line 81: Enter pytest.raises(ValueError, match='no rows'). The indented statements use this layout or test context.

Line 82: Load, validate and deduplicate the source, returning data and quality counts.

Line 85: Define test_feature_math_no_target_leak_and_zero_division(data). The indented body runs when this function is called.

Line 86: Assign row: make a separate copy so later edits do not alter the source object.

Line 87: Assign row.loc[0, ['views', 'likes', 'comments', 'watch_time_minutes']]: collect these values in the specified order.

Line 88: Assign features: select the approved inputs and calculate the two ratio features.

Line 89: Verify features.loc[0, 'engagement_rate'] == 0.15. A false condition fails this check rather than silently continuing.

Line 90: Verify features.loc[0, 'average_watch_minutes'] == 2.5. A false condition fails this check rather than silently continuing.

Line 91: Verify set(features.columns) == set(INPUTS + DERIVED). A false condition fails this check rather than silently continuing.

Line 92: Assign row.loc[0, TARGET]: store this literal value as a setting or initial value.

Line 93: Verify that both DataFrames agree in values and structure.

Line 94: Assign row.loc[0, 'views']: store this literal value as a setting or initial value.

Line 95: Verify add_features(row)[DERIVED].isna().all().all(). A false condition fails this check rather than silently continuing.

tests/test_project.py | lines 97-120

test_imputer_learns_only_fitted_rows: This test checks imputer learns only fitted rows. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_final_imputer_and_artifact_hashes: This test checks final imputer and artifact hashes. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

```
 97
 98  def test_imputer_learns_only_fitted_rows(data):
 99      train = data.head(100).copy()
100      pipeline = build_pipeline(LinearRegression()).fit(train[INPUTS], train[TARGET])
101      imputer = pipeline["preparation"].named_transformers_["numbers"]["imputer"]
102      expected = add_features(train)[NUMERIC + DERIVED].median().to_numpy()
103      np.testing.assert_allclose(imputer.statistics_, expected)
104      before = imputer.statistics_.copy()
105      other = data.iloc[100:120][INPUTS].copy()
106      other["likes"] = 1e9
107      pipeline.predict(other)
108      np.testing.assert_array_equal(imputer.statistics_, before)
109
110
111  def test_final_imputer_and_artifact_hashes(data, model, metadata):
112      train, validation, test = split_data(data)
113      development = pd.concat([train, validation])
114      expected = add_features(development)[NUMERIC + DERIVED].median().to_numpy()
115      imputer = model["preparation"].named_transformers_["numbers"]["imputer"]
116      np.testing.assert_allclose(imputer.statistics_, expected)
117      assert file_hash(utils.RAW_DATA) == metadata["data_sha256"]
118      assert file_hash(ROOT / "models" / "revenue_pipeline.joblib") == metadata["model_sha256"]
119
120
```

Line 98: Define test_imputer_learns_only_fitted_rows(data). The indented body runs when this function is called.

Line 99: Assign train: make a separate copy so later edits do not alter the source object.

Line 100: Assign pipeline: learn preparation and model parameters from the explicitly supplied fitting rows.

Line 101: Assign imputer: select the named field, columns or rows from the existing object.

Line 102: Assign expected: convert the selected values to a NumPy array.

Line 103: Verify numeric agreement within floating-point tolerance.

Line 104: Assign before: make a separate copy so later edits do not alter the source object.

Line 105: Assign other: make a separate copy so later edits do not alter the source object.

Line 106: Assign other['likes']: store this literal value as a setting or initial value.

Line 107: Apply the fitted pipeline to obtain revenue estimates.

Line 108: Verify that the arrays have exactly the same values.

Line 111: Define test_final_imputer_and_artifact_hashes(data, model, metadata). The indented body runs when this function is called.

Line 112: Assign (train, validation, test): separate observations into training, validation and test by video ID.

Line 113: Assign development: join the supplied DataFrames; here this forms development data.

Line 114: Assign expected: convert the selected values to a NumPy array.

Line 115: Assign imputer: select the named field, columns or rows from the existing object.

Line 116: Verify numeric agreement within floating-point tolerance.

Line 117: Verify file_hash(utils.RAW_DATA) == metadata['data_sha256']. A false condition fails this check rather than silently continuing.

Line 118: Verify file_hash(ROOT / 'models' / 'revenue_pipeline.joblib') == metadata['model_sha256']. A false condition fails this check rather than silently continuing.

tests/test_project.py | lines 121-144

test_predictions_match_saved_test_results: This test checks predictions match saved test results. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_model_reload_and_column_order: This test checks model reload and column order. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_unknown_category_safe_at_pipeline_level: This test checks unknown category safe at pipeline level. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

```
121  def test_predictions_match_saved_test_results(data, model):
122      test = split_data(data)[2]
123      saved = pd.read_csv(ROOT / "reports" / "test_predictions.csv")
124      predicted = model.predict(test[INPUTS])
125      np.testing.assert_allclose(predicted, saved["predicted"], rtol=1e-10)
126      scores = regression_metrics(test[TARGET], predicted)
127      recorded = read_json(ROOT / "reports" / "test_metrics.json")
128      for key in scores:
129          assert scores[key] == pytest.approx(recorded[key])
130      assert recorded["mae"] < recorded["mean_baseline"]["mae"]
131
132
133  def test_model_reload_and_column_order(data, model, tmp_path):
134      path = tmp_path / "copy.joblib"
135      joblib.dump(model, path)
136      restored = joblib.load(path)
137      sample = data.head(100)[INPUTS]
138      np.testing.assert_allclose(model.predict(sample), restored.predict(sample[INPUTS[::-1]]))
139
140
141  def test_unknown_category_safe_at_pipeline_level(data, model):
142      sample = data.head(3)[INPUTS].copy()
143      sample["category"] = "Unseen category"
144      assert np.isfinite(model.predict(sample)).all()
```

Line 121: Define test_predictions_match_saved_test_results(data, model). The indented body runs when this function is called.

Line 122: Assign test: select the named field, columns or rows from the existing object.

Line 123: Assign saved: read a CSV into a Pandas DataFrame.

Line 124: Assign predicted: apply the fitted pipeline to obtain revenue estimates.

Line 125: Verify numeric agreement within floating-point tolerance.

Line 126: Assign scores: calculate MAE, RMSE and R^2 for the supplied actual and predicted values.

Line 127: Assign recorded: read saved JSON metadata or metrics as ordinary Python objects.

Line 128: Repeat the indented steps for each key from scores.

Line 129: Verify scores[key] == pytest.approx(recorded[key]). A false condition fails this check rather than silently continuing.

Line 130: Verify recorded['mae'] < recorded['mean_baseline']['mae']. A false condition fails this check rather than silently continuing.

Line 133: Define test_model_reload_and_column_order(data, model, tmp_path). The indented body runs when this function is called.

Line 134: Assign path: calculate the shown arithmetic or combine the indicated collections.

Line 135: Serialize the fitted pipeline to the model file.

Line 136: Assign restored: restore the fitted pipeline from the trusted saved artifact.

Line 137: Assign sample: select the named field, columns or rows from the existing object.

Line 138: Verify numeric agreement within floating-point tolerance.

Line 141: Define test_unknown_category_safe_at_pipeline_level(data, model). The indented body runs when this function is called.

Line 142: Assign sample: make a separate copy so later edits do not alter the source object.

Line 143: Assign sample['category']: store this literal value as a setting or initial value.

Line 144: Verify np.isfinite(model.predict(sample)).all(). A false condition fails this check rather than silently continuing.

tests/test_project.py | lines 145-168

test_bad_prediction_inputs: This test checks bad prediction inputs. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_valid_and_missing_input_predictions: This test checks valid and missing input predictions. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_clean_export_is_complete_and_auditable: This test checks clean export is complete and auditable. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

```
145
146
147 @pytest.mark.parametrize("column,value", [("views", 0), ("views", 1e6),
148     ("likes", -1), ("comments", 48.5), ("watch_time_minutes", np.inf),
149     ("views", None), ("video_length_minutes", "invalid"), ("country", "XX")])
150 def test_bad_prediction_inputs(model, metadata, valid_input, column, value):
151     valid_input[column] = value
152     with pytest.raises(ValueError):
153         predict_revenue(model, valid_input, metadata)
154
155
156 @pytest.mark.parametrize("unknown", [[], ["likes"], ["comments"], ["watch_time_minutes"],
157                                      ["likes", "comments", "watch_time_minutes"]])
158 def test_valid_and_missing_input_predictions(model, metadata, valid_input, unknown):
159     for column in unknown:
160         valid_input[column] = None
161     prediction, notes = predict_revenue(model, valid_input, metadata)
162     assert np.isfinite(prediction) and prediction >= 0
163     assert len(notes) >= len(unknown)
164
165
166 def test_clean_export_is_complete_and_auditable():
167     cleaned = pd.read_csv(ROOT / "data" / "cleaned_youtube_data.csv")
168     assert len(cleaned) == 120000 and not cleaned.isna().any().any()
```

Line 147: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 150: Define test_bad_prediction_inputs(model, metadata, valid_input, column, value). The indented body runs when this function is called.

Line 151: Assign valid_input[column]: reuse the existing value named value.

Line 152: Enter pytest.raises(ValueError). The indented statements use this layout or test context.

Line 153: Validate a single input row and return its predicted revenue plus notes.

Line 156: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 158: Define test_valid_and_missing_input_predictions(model, metadata, valid_input, unknown). The indented body runs when this function is called.

Line 159: Repeat the indented steps for each column from unknown.

Line 160: Assign valid_input[column]: store this literal value as a setting or initial value.

Line 161: Assign (prediction, notes): validate a single input row and return its predicted revenue plus notes.

Line 162: Verify np.isfinite(prediction) and prediction >= 0. A false condition fails this check rather than silently continuing.

Line 163: Verify len(notes) >= len(unknown). A false condition fails this check rather than silently continuing.

Line 166: Define test_clean_export_is_complete_and_auditable(). The indented body runs when this function is called.

Line 167: Assign cleaned: read a CSV into a Pandas DataFrame.

Line 168: Verify len(cleaned) == 120000 and (not cleaned.isna().any().any()). A false condition fails this check rather than silently continuing.

tests/test_project.py | lines 169-192

test_displayed_range_endpoints_are_accepted: This test checks displayed range endpoints are accepted. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_all_app_pages: This test checks all app pages. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_app_valid_invalid_and_missing_predictions: This test checks app valid invalid and missing predictions. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

```
169     assert cleaned["likes_was_missing"].sum() == 6000
170     assert set(cleaned["partition"]) == {"train", "validation", "test"}
```

```
171
172
173 @pytest.mark.parametrize("column", NUMERIC)
174 def test_displayed_range_endpoints_are_accepted(model, metadata, valid_input, column):
175     for value in metadata["ranges"][column]:
176         valid_input[column] = value
177         prediction, notes = predict_revenue(model, valid_input, metadata)
178         assert np.isfinite(prediction)
179
180
181 @pytest.mark.parametrize("page", ["Overview", "Explore Data", "Predict Revenue", "Model Results"])
182 def test_all_app_pages(page):
183     app = AppTest.from_file(str(ROOT / "app.py"), default_timeout=40).run()
184     app.sidebar.radio[0].set_value(page).run()
185     assert not app.exception and not app.error
186
187
188 def test_app_valid_invalid_and_missing_predictions():
189     app = AppTest.from_file(str(ROOT / "app.py"), default_timeout=40).run()
190     app.sidebar.radio[0].set_value("Predict Revenue").run()
191     app.button[0].click().run()
192     assert app.metric[0].value.startswith("$") and not app.exception
```

Line 169: Verify cleaned['likes_was_missing'].sum() == 6000. A false condition fails this check rather than silently continuing.

Line 170: Verify set(cleaned['partition']) == {'train', 'validation', 'test'}. A false condition fails this check rather than silently continuing.

Line 173: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 174: Define test_displayed_range_endpoints_are_accepted(model, metadata, valid_input, column). The indented body runs when this function is called.

Line 175: Repeat the indented steps for each value from metadata['ranges'][column].

Line 176: Assign valid_input[column]: reuse the existing value named value.

Line 177: Assign (prediction, notes): validate a single input row and return its predicted revenue plus notes.

Line 178: Verify np.isfinite(prediction). A false condition fails this check rather than silently continuing.

Line 181: Apply the stated decorator: caching for the app, or fixture/parameterized behavior for tests.

Line 182: Define test_all_app_pages(page). The indented body runs when this function is called.

Line 183: Assign app: call AppTest.from_file(str(ROOT / 'app.py'), default_timeout=40).run with the shown arguments.

Line 184: Call app.sidebar.radio[0].set_value(page).run with the shown arguments.

Line 185: Verify not app.exception and (not app.error). A false condition fails this check rather than silently continuing.

Line 188: Define test_app_valid_invalid_and_missing_predictions(). The indented body runs when this function is called.

Line 189: Assign app: call AppTest.from_file(str(ROOT / 'app.py'), default_timeout=40).run with the shown arguments.

Line 190: Call app.sidebar.radio[0].set_value('predict revenue').run with the shown arguments.

Line 191: Call app.button[0].click().run with the shown arguments.

Line 192: Verify app.metric[0].value.startswith('$') and (not app.exception). A false condition fails this check rather than silently continuing.

tests/test_project.py | lines 193-215

test_app_filters_and_partial_date: This test checks app filters and partial date. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

test_missing_model_has_recovery_message: This test checks missing model has recovery message. Read the setup first, then the action and its assertions. Temporary files isolate failure tests from the delivered artifacts.

```
193     app.number_input(key="input_views").set_value(1000000)
194     app.button[0].click().run()
195     assert app.error and "outside" in app.error[0].value and not app.exception
196     app.number_input(key="input_views").set_value(10000)
197     app.multiselect[0].set_value(["watch_time_minutes"]).run()
198     app.button[0].click().run()
199     assert app.warning and app.metric and not app.exception
200
201
202 def test_app_filters_and_partial_date():
203     app = AppTest.from_file(str(ROOT / "app.py"), default_timeout=40).run()
204     app.sidebar.radio[0].set_value("Explore Data").run()
205     app.multiselect[0].set_value(["Gaming"]).run()
```

```
206      assert int(app.metric[0].value.replace(",", "")) < 120000
207      app.date_input[0].set_value((pd.Timestamp("2024-06-09").date(),)).run()
208      assert app.info and not app.exception
209
210
211  def test_missing_model_has_recovery_message(monkeypatch, tmp_path):
212      monkeypatch.setattr(utils, "ROOT", tmp_path)
213      app = AppTest.from_file(str(ROOT / "app.py"), default_timeout=40).run()
214      app.sidebar.radio[0].set_value("Predict Revenue").run()
215      assert app.error and "unavailable" in app.error[0].value and not app.exception
```

Line 193: Call app.number_input(key='input_views').set_value with the shown arguments.

Line 194: Call app.button[0].click().run with the shown arguments.

Line 195: Verify app.error and 'outside' in app.error[0].value and (not app.exception). A false condition fails this check rather than silently continuing.

Line 196: Call app.number_input(key='input_views').set_value with the shown arguments.

Line 197: Call app.multiselect[0].set_value(['watch_time_minutes']).run with the shown arguments.

Line 198: Call app.button[0].click().run with the shown arguments.

Line 199: Verify app.warning and app.metric and (not app.exception). A false condition fails this check rather than silently continuing.

Line 202: Define test_app_filters_and_partial_date(). The indented body runs when this function is called.

Line 203: Assign app: call AppTest.from_file(str(ROOT / 'app.py'), default_timeout=40).run with the shown arguments.

Line 204: Call app.sidebar.radio[0].set_value('explore data').run with the shown arguments.

Line 205: Call app.multiselect[0].set_value(['gaming']).run with the shown arguments.

Line 206: Verify int(app.metric[0].value.replace(',', '')) < 120000. A false condition fails this check rather than silently continuing.

Line 207: Call app.date_input[0].set_value((pd.timestamp('2024-06-09').date(),)).run with the shown arguments.

Line 208: Verify app.info and (not app.exception). A false condition fails this check rather than silently continuing.

Line 211: Define test_missing_model_has_recovery_message(monkeypatch, tmp_path). The indented body runs when this function is called.

Line 212: Call monkeypatch.setattr with the shown arguments.

Line 213: Assign app: call AppTest.from_file(str(ROOT / 'app.py'), default_timeout=40).run with the shown arguments.

Line 214: Call app.sidebar.radio[0].set_value('predict revenue').run with the shown arguments.

Line 215: Verify app.error and 'unavailable' in app.error[0].value and (not app.exception). A false condition fails this check rather than silently continuing.

21. Notebook execution helper

This command-line helper executes the notebook with the current Python environment and reliably shuts down its kernel afterward.

tests/run_notebook.py | lines 1-24

```
1  """Execute every notebook cell with the Python environment running this script."""
2
3  import asyncio
4  import sys
5  from pathlib import Path
6
7  import nbformat
8  from nbclient import NotebookClient
9  from jupyter_client import KernelManager
10
11 if sys.platform == "win32":
12     asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
13
14 project = Path(__file__).resolve().parents[1]
15 path = project / "notebooks" / "project_analysis.ipynb"
16 notebook = nbformat.read(path, as_version=4)
17 manager = KernelManager(kernel_name="python3")
18 manager.kernel_spec.argv = [sys.executable, "-m", "ipykernel_launcher", "-f", "{connection_file}"]
19 client = NotebookClient(notebook, km=manager, timeout=600,
20                         resources={"metadata": {"path": str(project / "notebooks")}})
21 try:
22     client.execute()
23     nbformat.write(notebook, path)
24     print(f"Completed all {len(notebook.cells)} cells without a cell error.")
```

Line 3: Import asyncio for Windows-compatible notebook event handling.

Line 4: Import sys for the current interpreter and module search path.

Line 5: Import Path from pathlib, providing portable file paths.

Line 7: Import nbformat for notebook file structure.

Line 8: Import NotebookClient from nbclient, providing fresh-kernel notebook execution.

Line 9: Import KernelManager from jupyter_client, providing kernel process management.

Line 11: Run the following indented branch only when sys.platform == 'win32' is true.

Line 12: Call asyncio.set_event_loop_policy with the shown arguments.

Line 14: Assign project: select the named field, columns or rows from the existing object.

Line 15: Assign path: calculate the shown arithmetic or combine the indicated collections.

Line 16: Assign notebook: call nbformat.read with the shown arguments.

Line 17: Assign manager: call KernelManager with the shown arguments.

Line 18: Assign manager.kernel_spec.argv: collect these values in the specified order.

Line 19-20: Assign client: call NotebookClient with the shown arguments.

Line 21: Try the following operation; a matching exception is handled by the following recovery branch.

Line 22: Run the notebook cells in a fresh kernel; an uncaught cell error fails execution.

Line 23: Display explanatory text or values in the current Streamlit container.

Line 24: Write the status or result to the terminal or notebook output.

tests/run_notebook.py | lines 25-27

```
25 finally:
26     if manager.has_kernel:
27         manager.shutdown_kernel(now=True)
```

Line 25: Run this cleanup whether the preceding operation succeeds or fails.

Line 26: Run the following indented branch only when manager.has_kernel is true.

Line 27: Stop the notebook kernel so it does not remain as a background process.

22. Launcher, dependencies and configuration

START_APP.bat is a Windows convenience wrapper. The Python files also run independently with the manual README commands. Its labels are jump destinations for success and failure handling.

```
@echo off
setlocal
cd /d "%~dp0"
if exist ".venv\Scripts\python.exe" goto install
py -3.12 --version >nul 2>&1
if not errorlevel 1 (
    py -3.12 -m venv .venv
) else (
    python -c "import sys; sys.exit(sys.version_info[:2] != (3, 12))" >nul 2>&1
    if errorlevel 1 goto python_missing
    python -m venv .venv
)
if not exist ".venv\Scripts\python.exe" goto setup_failed
:install
".venv\Scripts\python.exe" -m pip install -r requirements.txt
if errorlevel 1 goto setup_failed
".venv\Scripts\python.exe" -m streamlit run app.py --server.address 127.0.0.1
if errorlevel 1 goto app_failed
exit /b 0
:python_missing
echo Install 64-bit Python 3.12 from python.org, then run this file again.
pause
exit /b 1
:setup_failed
echo Setup did not complete. Check your Internet connection and the message above.
pause
exit /b 1
:app_failed
echo The app stopped. Read README.md for troubleshooting and alternative port commands.
pause
exit /b 1
```

Lines / command	Explanation
1-3	Hide command echo, localize batch settings and change to the launcher folder.
4-13	Reuse an existing local environment or locate Python 3.12 and create one. Redirect version-check output to avoid unnecessary noise.
14-19	Install the pinned packages, stop on setup failure, launch Streamlit locally and return a successful exit code when appropriate.
20 onward	Provide distinct messages for a missing interpreter, failed setup or failed app launch. pause leaves the message visible; exit /b 1 signals failure.

requirements.txt pins direct dependencies so training and model loading use compatible versions. Scikit-learn provides the algorithms, NumPy/Pandas the arrays and tables, joblib persistence, Streamlit the interface, Plotly interactive charts, Matplotlib/Seaborn saved figures, nbformat/nbclient/ipykernel notebook execution and pytest automated checks.

```
numpy==2.2.6
pandas==2.2.3
scikit-learn==1.7.2
joblib==1.5.2
streamlit==1.50.0
plotly==6.3.1
matplotlib==3.10.7
seaborn==0.13.2
nbformat==5.10.4
nbclient==0.10.2
ipykernel==6.30.1
pytest==8.4.2
```

requirements-lock.txt lists the full tested environment, including transitive and documentation dependencies. It is a reproducibility record; the direct requirements suffice for the application and its tests. A package listed transitively can still be used by the notebook runner, for example jupyter-client through ipykernel.

.streamlit/config.toml

```
[theme]
base = "light"
primaryColor = "#137F76"
backgroundColor = "#FAFAF6"
secondaryBackgroundColor = "#F0F3EE"
textColor = "#172E2C"
font = "sans serif"

[browser]
gatherUsageStats = false
```

The theme values set the primary accent, page background, secondary background and text color. The sidebar behavior itself is set in app.py. gatherUsageStats=false disables Streamlit usage-statistics collection. The configuration contains no credentials.

.gitignore

```
.venv/
__pycache__/
.pytest_cache/
.ipynb_checkpoints/
*.pyc
.DS_Store
.streamlit/secrets.toml
```

Ignore the local environment, compiled Python caches, pytest cache and notebook checkpoints. Keep any future Streamlit secrets out of Git. This prevents unnecessary machine-specific files and credentials from becoming part of the shared source.

23. Sources and final limitations

Project brief: https://docs.google.com/document/d/1zBCNNywwsDs3oEquJ7Q6Dwdt87_B5E-LVdzmjkOZOvw/edit

Supplied dataset: https://drive.google.com/file/d/1joozv6lXGgvnYZxbdKlV4VpVp2rbR_7s/view

EDA guide: https://docs.google.com/document/d/1tHiTU1X9UwXSLySpJ-FVCohlf_8xpXwa75vlK9S6wl8/edit

In-depth EDA reference: https://docs.google.com/document/d/1bzW_w-9m5WfyjVWvid2x8GR7C5mB4ONiMaqWzXTnQf0/edit

Model explanation guideline: https://docs.google.com/document/d/1gbhLvJYY7J73lu1g9c6C9LRJvYemiDOdRDAEMe632w8/edit

Submission instructions: https://docs.google.com/document/d/1FIltyOvXurieeuKG0n08uAKWWKNOdOW2X4y_7rXZ2nQ/edit

Streamlit references: https://docs.google.com/document/d/1aR3pUZFlCi8gicpF6aPHPESeFdOtGMlfob5PckresZk/edit

Pickling references: https://drive.google.com/drive/folders/1N8wD1OIQry1OAVTI8vpehoGM3Z4SGdHX

The project requirement and relevant written guides, sample code and notebooks were reviewed. The linked recordings were located but were not watched or transcribed end to end. No booking was submitted and no repository was published. Session notices conflict, so use the organizer's current dated schedule rather than embedding an old time in the app.

The source has no verified real-world provenance or representative coverage beyond its stated synthetic learning purpose. Before real-world use, collect representative data with stable definitions and evaluate on unseen channels and future dates. Re-run the tests after changing dependencies, data or code. The checked release demonstrates the intended learning workflow without claiming universal accuracy or a guarantee against every future software error.